The Thiele Machine

A Complete Account from First Principles:
Structural Entitlement, Accountable Computation, and No Free Insight

Devon Thiele

April 2026

Abstract

I started building this in January 2025. I had no CS degree. I was trying to make a 3D renderer. What I ended up with, fifteen months later, is a new model of computation with machine-checked proofs, a hardware implementation, and a result I did not expect: the machine's internal honesty condition for a 2-player game turns out to be exactly equivalent to a polynomial constraint system from quantum information theory — specifically, positive semidefiniteness of the NPA moment matrix for zero-marginal CHSH correlations. No physics assumed. Pure algebra.

The central idea is simple. A Turing machine tracks registers, memory, and time. It cannot see whether a computation has become *entitled* to use a structural shortcut — a decomposition, a verified invariant, a certified fact about its own state. The Thiele Machine makes that entitlement first-class and cost-bearing. There is a global counter called μ that starts at zero and never goes down. Any computation that ends in a certified structural claim must have paid at least 1 into μ. That is No Free Insight. It is a conservation law, proved in Coq from the machine's own step function.

46 opcodes. All in hardware. All with formal correctness proofs — 30 fully unconditional, 6 requiring valid arguments, 7 unconditional for any state reachable from hardware reset, 3 with a semantic boundary. A conservation law. A unique canonical cost measure. A complexity hierarchy. A biconditional between the machine's honesty condition and the zero-marginal NPA polynomial realizability condition. Every proof compiles or it doesn't. That is the point.

Contents

1 Let me be straight with you

No CS degree. No math degree. No physics degree. I barely knew what programming was. And I was not trying to build a formally verified CPU.

I was trying to build a 3D renderer.

I had read about category theory and thought it was the right mathematical structure for a rendering pipeline that could not be wrong. Objects, morphisms, a rendering functor. Get triangles on screen using math that enforced correctness by design. I built it. At 2 AM on January 22, the morphisms were being applied correctly. That was supposed to be the end of the project.

Sixteen days in, I realized the categorical framework I had built for rendering also ran physics simulations without any changes. I plugged Newton's laws into the same category structure and it ran. I did not plan that. I discovered it. I went from debugging why a pixel was not appearing to realizing I had built something that described physics too.

I followed that question for eight months. Rendering became physics became formal computation became Coq proofs became hardware. Every step revealed another layer I did not know existed. I did not know what a Turing machine was when I started. I did not know what Coq was. I learned each thing by needing it for the next step.

I used large language models as implementation tools throughout. The tool was useless for most of the early work because nothing like this existed: no reference implementation, no prior work to hand it, no Stack Overflow answer. I had to build all the context from scratch before the tool became useful, and that took months. Once the context was there, it became a force multiplier. The ideas, the architecture, the math, the direction: mine. The proofs compile or they don't, and I checked every one of them.

I'm telling you this because it matters for how I think. I don't come from a background where anyone told me "this is how it works, trust us." I don't have professors or colleagues whose authority I can lean on. When I encounter a claim, I either find the proof or I don't believe it. That's not a methodology I adopted. It's how I think. It's the only way I know.

Everything in here gets built from scratch. Not as a courtesy. Because I don't believe I understand something until I've built it, and I built this document the same way I built the machine.

If you are an expert and you find this review of basics annoying, skip to Section 8. The technical content starts there. But if you find anything in the basics wrong, I want to know. Nothing here is correct just because I wrote it.

What this project is. The Thiele Machine is a model of computation where structural

entitlement is a first-class state component. By entitlement I mean the machine is not merely storing data; it is allowed, by its own checked trace, to use a decomposition, relation, invariant, certificate, or narrowed feasible set as part of the computation. No Free Insight is the first theorem that falls out: a computation cannot end in a stronger certified claim than it started with unless the strengthening passed through the machine's explicit cost ledger. This is formalized in Coq and machine-checked. The ledger is called μ. It never goes down. If you go from uncertified to certified, μ went up. Scope: raw μ is a receipt that certification work was paid for. It is not, by itself, a universal depth score for the meaning of the claim. The stronger semantic reading only applies when the certification comes with an explicit narrowing witness. That distinction is not cosmetic. It is the hinge between an honest cost ledger and an overclaim.

What this project is not. It is not a claim that I solved P vs NP. It is not a theory of everything. It is not a claim that computation is physics or that physics is computation. The CHSH algebraic result is pure math. The Landauer connection is motivation. That is the full extent of the physics content here.

2 The Turing Machine: what it can do and what it cannot see

The Turing machine. What it is, exactly, and where it falls short. Not vaguely falls short. Precisely.

2.1 What a Turing machine is

Alan Turing described his machine in 1936 to make precise the intuitive notion of "algorithm." The machine is this:

- An infinite tape. Think of it as an infinitely long strip of paper, divided into cells. Each cell holds one symbol from a finite alphabet, say $\{0, 1, \texttt{blank}\}$.
- A read/write head. It sits on one cell at a time. It can read the symbol in that cell, write a new symbol over it, and move one step left or one step right.
- A finite set of states. Call them $Q = \{q_0, q_1, \ldots, q_n\}$. The machine is always in exactly one state. q_0 is the start state. Some states are designated "halting" states.
- A transition table. For each combination of (current state, symbol under the head), the table says: write this symbol, move this direction, go to this new state.

That is the entire machine. Nothing else.

Here is the remarkable thing: this simple machine can compute *anything that can be computed by following a definite procedure*. Every algorithm that has ever been written, in every programming language that has ever existed, can be translated into a Turing machine. This is the Church-Turing thesis. The original claim (Church 1936, Turing 1936) is specifically about *effective calculability*: anything that can be computed by following a mechanical, finite, step-by-step procedure, a Turing machine can also

compute. The thesis has never been falsified within mathematics and computation theory.

The claim about physical hardware is a related but separate conjecture, sometimes called the *physical Church-Turing thesis*: that every physical computing device is computationally equivalent to a Turing machine. This is an empirical claim about physics, not proven mathematics. So far, every physical computing device built turns out to be equivalent to a Turing machine in what it can compute. Your laptop, your phone, every server in every data center: all of them are, in the relevant theoretical sense, Turing machines. They can compute exactly the same class of things. The only confirmed difference is speed.

2.2 The blind spot

Here is the precise limitation. The transition table sees exactly two things: the current state q and the symbol under the head. That's it.

The machine cannot ask: "Is this tape sorted?" It has to read every cell.

The machine cannot ask: "Does this graph have two disconnected components?" It has to run a traversal.

The machine cannot ask: "Are these two parts of the input independent of each other?" It has to check.

The Turing machine's view of its tape is LOCAL. One cell at a time. It has no primitive way to see global structure. It can COMPUTE global structure, but computing it and seeing it are different things. To compute that a list is sorted, the machine has to execute a sorting-detection algorithm that touches every element. It pays in time and space for every structural fact it learns.

I spent months staring at this before it clicked. The Turing Machine isn't broken. It's blind by design. It's like trying to find your way through a maze by only ever looking at the floor tile you're standing on. You *might* do it — if the maze has an exit and if your search strategy eventually reaches it. Some mazes have no exit, and some search strategies loop forever. That is the halting problem: the machine cannot in general tell from the floor tile whether it is making progress or going in circles. But if the maze does terminate, you are paying for every wrong turn and every part already explored. Someone with a map doesn't pay that cost.

The Thiele Machine does not magically give the map. "The map" here is a concrete thing: a certified partition graph with proven structural properties: a checked decomposition into modules, a validated formula, an asserted morphism with provenance. The Thiele Machine represents the specific moment a computation becomes entitled to use one of these, and it makes that entitlement auditable: there is always a μ-ledger entry showing when the certification was paid for. The map is not free.

2.3 Classical complexity ignores structural cost

Standard complexity theory measures two things: time (number of steps) and space (number of cells or registers used). It says nothing about the cost of knowing that the input has structure.

Consider the satisfiability problem (SAT): given a logical formula ϕ over n boolean variables, is there an assignment that makes it true? In the worst case, a blind machine has to check all 2^n assignments. That's what makes SAT hard.

Now suppose ϕ decomposes cleanly: $\phi = \phi_1 \wedge \phi_2$, where ϕ_1 and ϕ_2 share no variables. You can solve ϕ_1 on its own (2^{n_1} work) and ϕ_2 on its own (2^{n_2} work). Total work: $2^{n_1} + 2^{n_2}$ instead of 2^n. That is an exponential improvement.

But that improvement only applies if you *know* the decomposition exists. And the Turing machine model assigns zero cost to knowing it. The structure is either there or it isn't, and you either see it or you spend time finding it, but the act of certifying "I know this decomposition is valid" is not tracked, not paid for, not audited.

The Thiele Machine says: that act belongs inside the computation. Here is the structural object. Here is the checked transition that makes the object admissible. Here is where the cost shows up in the trace. Here is how to verify that the cost was actually paid. This is the structural-entitlement reading of No Free Insight.

3 What structure is

"Structure" is one of those words everyone uses and nobody defines. The whole machine rests on what it means, so here is the definition.

3.1 Partitions

Start with the simplest case. Take any set S and divide it into labeled buckets that don't overlap and cover everything. That is a partition.

Formally: a partition of S is a collection of disjoint non-empty subsets $S_1, S_2, \ldots, S_k$ such that $S_1 \cup S_2 \cup \cdots \cup S_k = S$. Each S_i is called a module or block or cell, depending on the context.

Now here is the key thing: the partition IS information. Knowing that element x belongs to block S_1 and element y belongs to block S_2 tells you something real. It tells you x and y are in different groups. If you didn't know the partition, you didn't know that. If you know the partition, you can look it up in constant time.

In a computational context, partitioning the memory or register state means saying: "These addresses belong to module A, those addresses belong to module B, and these two modules are independent." That is structural knowledge. A Turing machine has no way to represent this directly. It has to recompute independence from scratch every time it needs to know it. The Thiele Machine stores the partition as part of its state,

audits the cost of building it, and enforces that building it was paid for.

3.2 Axioms on modules

A partition alone is just a labeling. What makes it structural knowledge is when you attach *constraints* to the modules.

First, two definitions I need.

A **predicate** is a statement about a thing that is either true or false. "$x > 5$" is a predicate about a number x. "This memory region contains a sorted list" is a predicate about a memory region. Predicates are the basic unit of making a claim about something.

A **logical formula** over boolean variables is an expression built from variables (each either true or false) connected by AND ($\wedge$), OR ($\vee$), and NOT ($\neg$). Example: $(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$ — this formula is true when either x_1 or x_2 is true AND when either x_1 is false or x_3 is true. A satisfying assignment is a specific choice of true/false for each variable that makes the whole formula evaluate to true. An unsatisfying assignment makes it false.

With those in hand: you attach constraints to the modules. For example: "Module A's memory region satisfies the predicate Φ_A," where Φ_A might say "this region is a valid binary heap" or "all values in this region are between 0 and 100" or "this region encodes a satisfying assignment to the formula stored in module B."

These constraints are called axioms. They are written in a formal logic.

I'll come back to the exact encoding in the ISA section, but the rough picture is: the machine reads a formula from memory, checks it against a certificate (a witness that the formula is satisfiable — but also a witness that it's not a tautology), and either accepts or traps. The piece of hardware that does that checking is called the Logic Engine. It's a finite-state machine built into the chip — a small dedicated unit that walks through the formula word by word and verifies the witnesses without calling out to any external solver. If validation succeeds, the checked constraint package can be used by the later certification path. If validation fails, the machine traps and sets an error flag. The cost is charged either way.

The crucial point: you cannot add an axiom to a module for free. Adding an axiom means the machine paid to certify it. The axiom is evidence, not assertion.

3.3 Structural gain

A trace has "structural gain" if it starts in a state where some structural fact is not certified and ends in a state where it is. The classic example: a trace that starts with an uncertified partition and ends with a certified satisfying-assignment witness has undergone structural gain.

The Thiele Machine's core question is: where, in the trace, did the structural gain enter? Not merely "did the gain happen." That is visible from inspecting the start and

end states. The question is which instruction, at which step, introduced the certified structural knowledge.

The answer is always: an instruction that costs $\mu \geq 1$. That is No Free Insight.

4 Structural entitlement

The center of the project is not just "certification has a price." That is true, but it is the first conservation law, not the whole picture.

The deeper object is *structural entitlement*. A computation has structural entitlement when the state does not merely contain data, but contains an admissible structural fact that later steps are allowed to use: a partition, a decomposition, a morphism, a constraint, a witness, or a narrowed feasible set. The difference matters. A raw bit pattern can be copied. A certified decomposition can justify solving two smaller problems instead of one large one. A raw morphism ID can be forged in a register. An asserted morphism in the graph has *provenance* — meaning you can trace it back to the specific MORPH and COMPOSE instructions that built it; the machine's execution history is the chain of evidence. A formula in memory is text. A checked formula with a satisfying assignment and a falsifying assignment is admissible narrowing: it proves the constraint is real, not a tautology, and rules out at least one state.
Definition 4.1 (Structural entitlement)**.** A state has structural entitlement to a claim P when three things are present:

- a structural object in the machine state, such as a partition, morphism, witness counter, or formula package;
- a checked relation between that object and the claim P;
- a trace-visible certification or assertion event that makes later use of P admissible.

Without the third part, the object may exist as raw data, but the computation has not earned the right to use it as certified structure.

The pieces connect as follows.

4.1 The precise vocabulary

Every term below is an exact Coq definition, not a metaphor. Every theorem that follows has been machine-verified. I'm not stamping each one individually with "machine-checked" — assume all of them are.

Feasible set. A feasible set Ω is a concrete, finite list of Thiele Machine states. In the Coq code, the type is literally `list VMState`. The interpretation: Ω is the set of machine states the computation might be in, given everything that has been checked and certified so far. Before any structural knowledge is certified, Ω can contain many states. After certifying a constraint, Ω shrinks to only the states consistent with that

constraint. A feasible set is not an abstract notion of "possible worlds." It is a list of actual machine states.

Prior and posterior. The *prior* Ω is the feasible set before an observation or structural certification event. The *posterior* Ω' is the feasible set after. The posterior is always a subset of the prior: $\Omega' \subseteq \Omega$. If the posterior is a *strict* subset ($|\Omega'| < |\Omega|$), some states have been ruled out. The observation eliminated them. The picture is: you start with a universe of possibilities. You run an experiment. You learn which outcomes are consistent with the result. The states that are no longer consistent get thrown out. What remains is the posterior. Here, "learning something" means the machine certified a structural claim that is false in those eliminated states, so they can no longer be the actual machine state.

Distinguishing observation. A distinguishing observation is a function that returns different values on the eliminated states (in Ω but not in Ω') versus the remaining states (in Ω'). Formally: there exists at least one eliminated state $s_w \in \Omega \setminus \Omega'$ such that the observation function outputs a value on s_w that no state in Ω' produces. The observation is the "test" that reveals the structure. If you cannot name such a function, you cannot show that the posterior genuinely excludes something the prior allowed.

Strictly stronger predicate. A predicate P_1 is strictly stronger than P_2 if P_1 accepts a strict subset of what P_2 accepts. Formally in Coq: $P_1 \leq P_2$ (whenever P_1 is true, P_2 is true too) AND there exists an observation where P_1 is false but P_2 is true. The word "stronger" means more restrictive: P_1 makes a harder claim, rules out more states as invalid. If P_2 is "the state is in Ω" and P_1 is "the state is in $\Omega' \subsetneq \Omega$", then P_1 is strictly stronger — it demands more, and anything that satisfies P_1 also satisfies P_2 but not vice versa. This is the opposite of everyday use of "stronger": a stronger claim in this logical sense is harder to satisfy, not more impressive-sounding.

Structure-addition event. A structure-addition event is a single execution step where the assertion register `csr_cert_addr` transitions from 0 to nonzero. Precisely: s.`vm_csrs.csr_cert_addr` $= 0$ before the step and (`vm_apply` s i).`vm_csrs.csr_cert_addr` $\neq 0$ after. This only happens via a successful MORPH_ASSERT with a non-empty property string, which stores the ASCII checksum of that property in `csr_cert_addr`. Not "structural knowledge was acquired" in some vague sense. Precisely: this specific field flipped from zero, at this step, by MORPH_ASSERT asserting a non-empty property against an existing morphism.

Supra-certification (`has_supra_cert`). The machine has reached supra-certification when `csr_cert_addr` $\neq 0$. The name distinguishes it from `vm_certified` (the top-level flag set by the CERTIFY opcode). Supra-certification specifically tracks the MORPH_ASSERT channel: has a morphism property been asserted and its checksum registered? A machine can have `vm_certified = true` without

`has_supra_cert` (if it used only CERTIFY and not MORPH_ASSERT, since CERTIFY never touches `csr_cert_addr`). Both are positive-cost certification channels; they record different things.

Decision tree. A decision tree is an explicit witness to *how* the trace's observations branch over the state space. At each node, the tree records: here is an observation, these states passed it, these states failed it. The leaves record which states ended up in the posterior. The tree is a proof object, not a runtime structure: it exists to demonstrate that the transition from prior to posterior was the result of a definite sequence of binary tests rather than a magic jump. The Coq requires you to construct it explicitly as a witness; you cannot claim a shortcut is sound without handing over the actual branching structure that produced the posterior.

Posterior-representative reduction. Given a prior state $s \in \Omega$, a posterior-representative reduction provides a function mapping s to a representative $s' \in \Omega'$. This proves that every prior state corresponds to something in the posterior. The purpose: it shows the posterior was not reached by arbitrarily discarding states, but by a coherent mapping where each prior state is "accounted for" by a posterior representative. Without this, the narrowing could in principle be fake: you could claim the posterior is small by just deleting states without any principled connection to the prior.
Theorem 4.1 (Classical blindness as quotient)**.** *The four-field forgetful map from Thiele states to Turing-style snapshots is a many-to-one quotient. Its kernel is exactly equality on PC, μ, registers, and memory, so it can identify states that differ in certification, witness counters, or graph structure. The six-field `shadow_proj` keeps `vm_certified` but still forgets morphism structure; no function of that shadow alone can separate the morphism witness states.*

If your machine state only records registers, memory, PC, error bit, and maybe μ, you have already thrown away part of what matters. You have collapsed states that differ on the inside into a single raw state that looks the same from the outside. That is not a philosophical complaint. It is a theorem about a projection map.
Theorem 4.2 (μ-ledger necessity, complete independence classification)**.** *I write $X \perp Y$ to mean: knowing Y tells you nothing about X. There exist two reachable machine states with identical Y-values but different X-values. Not just from one starting point. From every starting point the machine can reach. Five such independence results hold simultaneously: (1) $\mu \perp (\texttt{mem}, \texttt{regs}, \texttt{pc})$; (2) $\texttt{certified} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc})$; (3) $\texttt{certified} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu)$; (4) $\mu \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \texttt{certified})$; (5) $\texttt{vm\_graph} \perp (\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu, \texttt{certified})$. The three non-classical components—μ, `vm_certified`, `vm_graph`—are each irrecoverable from any combination of the others plus classical state. The projection $(\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu, \texttt{certified})$ is the minimal complete extension for ledger semantics: dropping either μ or `vm_certified` loses the ability to determine the other. The separation holds starting from every reachable machine state, and no program prefix of any length collapses it.*

The ledger is not just a counter someone forgot to derive. There are explicit witness states with identical Turing/RAM-visible state and incompatible receipts—not just from one starting point, but from every state the machine can reach. Cost, certification, and graph structure are three independent bookkeeping dimensions. A model that drops any of them cannot determine the full structural semantics internally.

Theorem 4.3 (Latent μ: the cost is intrinsic to computation)**.** *The total μ accumulated by running any instruction sequence equals the sum of instruction costs, independent of every aspect of machine state (vm_graph, vm_regs, vm_mem, vm_pc, vm_certified, starting μ). Formally:*

$$\mu_{final} = \mu_{initial} + \sum_{i \in prog} \texttt{instruction\_cost}(i)$$

The right side is the sum of costs over every instruction in the program. It does not depend on the starting state at all. Two machines running the same program from any two starting states accumulate the same additional μ — the delta depends only on the program, not the starting state. Any instruction-consistent accounting system that assigns zero to the starting state assigns exactly `trace_total_cost` *to the result — there is no alternative honest accounting. Any program containing a positive-cost instruction accumulates strictly positive μ — there is no free computation.*

The μ-cost is not a feature of the Thiele Machine. It is a property of the instruction sequence itself. Replace the Thiele Machine's μ-counter with any other honest accounting and you get the same number. A Turing machine running the same program paid exactly this cost on every execution—it simply had no register to store the receipt. The ledger does not create the cost. It reveals what was always there.

Theorem 4.4 (Feasible narrowing becomes predicate strengthening)**.** *If a posterior feasible set Ω' is a strict subset of a prior feasible set Ω, and there is an observation that distinguishes a removed witness state from all posterior states, then the membership predicate for Ω' is strictly stronger than the membership predicate for Ω.*

This is the bridge from "the search space got smaller" to "the computation now accepts a stronger claim." The predicates are not fake constants. They are built from membership in the feasible sets through an observation function.

Theorem 4.5 (Strengthening requires structure addition)**.** *If a trace starts with no certification address, ends certified for a strictly stronger predicate, and the certification is tied to the machine's supra-certification channel, then the run contains a structure-addition event.*

This is the formal answer to "where did the new admissible structure enter?" It did not appear from arithmetic alone. It entered through a trace-visible structure-addition event.

Theorem 4.6 (Universal certification cost)**.** *For any abstract certification system with a state type, instruction type, step function, cost function, and certification predicate, if every single-step transition from uncertified to certified costs at least 1, then every trace from uncertified to certified has total cost at least 1.*

This last theorem is important because it is not about the Thiele ISA. It says the conser-

vation law is forced for any system once the system admits a real certification predicate and a sound cost rule for certification transitions. If a model lets the certification predicate flip at zero cost, then it has free certification by construction. If it refuses to represent certification at all, then it cannot talk about structural entitlement internally. It can only smuggle that entitlement in from outside the model.

Theorem 4.7 (Structural entitlement representation)**.** *For any explicit witness of structural entitlement consisting of:*

- *a strict feasible-set narrowing* $\Omega' \subsetneq \Omega$*;*
- *an observation function that distinguishes at least one eliminated witness state from all posterior states;*
- *a certified posterior predicate built from membership in* Ω'*;*
- *a decision tree* realized by the trace*: meaning the trace's actual sequence of observations matches the tree's branching structure — each branch in the tree corresponds to a test the execution actually performed; and*
- *a posterior-representative reduction tying the prior states to posterior representatives,*

the posterior predicate is strictly stronger than the prior predicate, the trace contains a structure-addition event, and the quantitative narrowing is bounded by the paid ledger:

$$\log_2^{\uparrow} |\Omega| - \log_2^{\uparrow} |\Omega'| \leq \Delta\mu.$$

($\log_2^{\uparrow}$ *is the ceiling of the base-2 logarithm: the smallest integer* $\geq \log_2$ *of the argument. It counts how many binary questions you need to locate a state in the set. The ceiling rounding is conservative: it can only make the bound harder to satisfy, never easier.)*

This theorem does not quantify over the English word "structure." It defines the witness class: strict narrowing, distinguishability, certification, and an explicit tree/fiber representative for the reduction. For every member of that class, structural entitlement is not metadata and not a slogan. It is predicate strengthening, it requires structure addition, and its quantified narrowing is charged to μ.

Theorem 4.8 (Every sound structural shortcut lands here)**.** *A* `SoundStructuralShortcut` *is a package containing exactly the receipts a shortcut must provide to count as sound: a prior feasible set, a posterior feasible set, strict narrowing, a distinguishing observation, a certified posterior predicate, a realized decision tree, and a posterior-representative reduction. For every such package, the structural-entitlement representation theorem applies. The shortcut lands in strictly stronger posterior knowledge, a structure-addition event, and the* $\Delta\mu$ *lower bound.*

This is the formal version of "it has to." Once a shortcut is no longer just a human story but a sound object the machine may use, it has to provide the data that makes the shortcut admissible. In this framework, that data is exactly the structural-entitlement witness. If someone says symmetry, factorization, modularity, independence, or

decomposition gave them the shortcut, the next question is no longer mystical. Show me the prior set, the posterior set, the observation that rules something out, and the representative/tree witness. If they can show that, the theorem applies. If they cannot, then the structure is still outside the computation.

4.2 What narrowing buys

The machine also has a concrete structural-advantage demonstration. `StructuralAdvantage.v` studies a factored search problem. The blind program has all instruction costs set to 0 and searches the flat space. The sighted program pays two receipt-visible events, each `EMIT "." 0`, for total cost 18, and searches two factors separately. The file proves the exact cost formulas and the arithmetic fact that the N^2 versus $2N$ gap eventually dwarfs the constant μ cost. The executable tests check the measured VM behavior on representative sizes.

There are two nearby claims here and it matters to keep them separate. The original `EMIT`-only $N = 1$ witness closes the observation layer: it proves posterior admissibility as `CertifiedObs` (a Coq type for "a certified observation has been recorded," meaning the machine has observed and recorded a result but has not yet activated a cert channel). It does not close the stronger bridge. The run does not end in `has_supra_cert`, and in the semantic CSR-transition sense it does not realize structure addition. A minimally extended witness does close the full theorem honestly: keep the factorized search exactly as it is, then append `PNEW`, `MORPH_ID`, and `MORPH_ASSERT`. That suffix is enough to fire the concrete certification channel, so the extended trace lands in the full structure-addition theorem.

That example is not a P vs NP result. It is a small, honest witness for the thing this machine is meant to track: certified structure can change which computation is admissible, and the cost of acquiring that admissibility is not the same as the time saved by using it.

4.3 The boundary

What I still do not have is the open translation theorem: whether every human informal story about symmetry, modularity, factorization, independence, or decomposition can always be packaged as a `SoundStructuralShortcut`. That would be a full informal-to-formal representation theorem for structural use, and it is stronger than what's proved here. It also sets the standard for the work: if a claimed shortcut can be pushed down into explicit witnesses, it has to be pushed to bedrock. If it can still be pushed, push again. Stop only where the proof honestly stops.

I've proved the hard internal theorem for the explicit class, and proved the boundary in both directions on a nearby concrete witness. The original `EMIT`-only factorized-search trace closes only the observation layer: the posterior predicate is certified observationally, but the run does not reach `has_supra_cert` and does not realize semantic structure addition. A minimally extended trace, with the factorized search

unchanged and only a post-search suffix `PNEW ; MORPH_ID ; MORPH_ASSERT`, does close the whole chain: stronger predicate, structure addition, and the $\Delta\mu$ lower bound. The unconditional Shannon-style statement for every deterministic single trace is still explicitly rejected in the code. The bridge needs a whole-tree, per-branch-preimage, or distributional witness — something that accounts for the full structure of how the search splits, not just a single execution path. That is not a weakness in the proof. It is where the proof stopped lying.

So the honest thesis is this: once structural entitlement is internalized with explicit witnesses, free certified strengthening is impossible; classical shadows lose entitlement-relevant distinctions; and feasible-set narrowing has a formal path into predicate strengthening and a ledger lower bound. The broader project is to show how many real structural shortcuts can be compiled into those witnesses.

5 What insight is, and why it cannot be free

In this machine, insight has a precise meaning. Three kinds of observation:

5.1 The three tiers of observation

Not all observations are the same. The taxonomy is proven in Coq. File: `WitnessInsightGeneral.v`.

Tier 0: Raw observation (always free). A raw observation records data without committing to any structural interpretation. The clearest example in the Thiele Machine is a CHSH trial (see Section 14): you run an experiment, you record the outcome in a counter, and nothing structural changes. The counter goes up. The cert channel is not touched. This is free. Cost: 0.

Tier 1: Certified structural insight (always costs ≥ 1). The machine has two recorded certification channels. One is the formula/certificate address channel, `csr_cert_addr`. The other is the top-level flag, `vm_certified`. Any instruction that turns either channel from "off" to "on" must have cost ≥ 1. This is proven in Coq (`certification_requires_positive_mu`). One thing to be precise about: in the current hardware-aligned step relation, LASSERT checks a constraint package and charges for it, but LASSERT itself does not directly mutate the cert channels. The concrete state-changing instructions are CERTIFY (sets `vm_certified = true`) and MORPH_ASSERT (sets `csr_cert_addr` nonzero). LASSERT performs the validation that might precede MORPH_ASSERT or CERTIFY. `AbstractNoFI.v` reasons about the broader certification/revelation class — which includes LASSERT, EMIT, REVEAL, LJOIN, READ_PORT, CERTIFY, and MORPH_ASSERT — as a positive-cost policy: all of them cost ≥ 1. The distinction matters: LASSERT checking a formula is part of the work chain, even if the formal cert-channel flip happens via the subsequent CERTIFY or MORPH_ASSERT.

Tier 2: Certified binary-game witness insight (always costs ≥ 1). The sharpest case: a

machine that is certified AND whose CHSH statistics show a violation of the classical bound ($S > 2$; more on CHSH in Section 14). Getting certified is already a Tier 1 event (cost ≥ 1). Getting the statistics to show a violation means the data can't be explained by any shared random strategy; this is proven in `CHSHStatisticalBridge.v`. Together: certified game-violation evidence costs at least 1 to acquire, over any trace of any length.

5.2 Why insight cannot be free

The proof:

Suppose a machine starts uncertified and ends certified. Somewhere in that trace, a specific opcode fired that flipped one of the certification registers from off to on. The two concrete cert-channel instructions are: CERTIFY (which sets `vm_certified = true`) and MORPH_ASSERT when the morphism exists (which sets `csr_cert_addr` to the ASCII checksum of the property string — nonzero for any non-empty property string). These are the only instructions that change those fields. MORPH_ASSERT with an empty property string or a missing morphism does not activate the cert channel, though it still costs $S(\delta) \geq 1$. This is proven by case analysis over all 46 opcodes — every other instruction leaves those fields unchanged. So: the cert flip happened at a CERTIFY or MORPH_ASSERT step. Look at that exact step. That step, by the semantics of the machine, has cost $S(\delta) \geq 1$, because both CERTIFY and MORPH_ASSERT are in the mandatory-floor class. So at that step: $\mu_{\text{after}} = \mu_{\text{before}} + S(\delta) \geq \mu_{\text{before}} + 1$. And because μ never decreases (each instruction adds a non-negative cost), $\mu_{\text{before}} \geq \mu_{\text{initial}}$. Therefore $\mu_{\text{final}} \geq \mu_{\text{after}} \geq \mu_{\text{before}} + 1 \geq \mu_{\text{initial}} + 1$.

This is not a philosophical argument. It is a structural property of the step function. The Coq proof checks it mechanically. The proof does not say "intuitively, certification should cost something." It says: here is the step function definition, here is the cost semantics, here is the monotonicity lemma, here is the induction on the trace. QED. The deeper reason is this: if you could certify structure for free, the certification would be meaningless. A receipt that any machine can produce at zero cost proves nothing. The whole point of the μ ledger is that it is unforgeable. You cannot accumulate certified structural knowledge without a trace of cost.

But cost alone is not yet semantic depth. A machine can spend resources checking a claim that turns out to be shallow. So in the tightened architecture, LASSERT does not count as structural certification merely because a formula parses and one satisfying assignment exists. It must also carry a non-triviality witness: one satisfying assignment and one falsifying assignment. That proves the asserted constraint excludes at least one state and therefore genuinely narrows the feasible set. The receipt proves spend; the witness proves the spend bought an actual narrowing.

6 The μ ledger

μ (mu) is the global structural cost ledger of the Thiele Machine.

6.1 What mu is

μ is a natural number attached to every machine state. It starts at 0. It never decreases.

Every instruction carries a cost parameter (δ) in its encoding. The machine adds that cost to μ when the instruction executes. Here is how δ is used in practice:

For **pure compute programs**, meaning programs that do arithmetic, load and store values, and jump, every instruction carries $\delta = 0$. The program I use as the concrete comparison case for the Turing Strictness theorem (`blind_program` in `StructuralAdvantage.v`) has this property: all instructions at $\delta = 0$, and the proof that its total μ-cost is zero is a direct calculation. For these programs, μ stays at 0 unless a positive-cost instruction fires.

For **certification and revelation instructions** (CERTIFY, LASSERT, MORPH_ASSERT, EMIT, REVEAL, LJOIN, READ_PORT): the cost is at least 1, regardless of δ. Even $\delta = 0$ still costs 1. Here is why that floor is forced, not chosen.

When CERTIFY fires, it commits the machine to a certified state: `vm_certified` flips from false to true. That commitment is irreversible — μ never goes down, and the certificate cannot be taken back. Committing to a structural fact is information-theoretically equivalent to erasing the "uncertified" possibility from your state space. By Landauer's argument from the previous section, erasing information has a minimum cost. The minimum honest cost of an irreversible certification step, as a function only of the programmer-supplied δ, is $\delta + 1$: the δ the programmer declares plus the mandatory 1 for the irreversible commit. That is $S(\delta)$.

Why $+1$ and not $+2$ or $+0.5$? Because $+1$ is the minimum that makes certification impossible for free. $\delta + 0$ would allow $\delta = 0$ to certify for free. $\delta + 2$ would be overcharging. $\delta + 1$ is the unique honest minimum: you can always choose $\delta = 0$ and pay exactly 1; you can choose $\delta > 0$ and pay more. The proof that $S(\delta)$ is the only consistent cost floor is in `MuCostDerivation.v` (`cost_necessity` + `cost_uniqueness`).

Throughout this document, $S(n)$ means $n + 1$ (Peano successor). $S(\delta) = \delta + 1 \geq 1$ always. EMIT, REVEAL, and READ_PORT go further: their cost also grows with the information content they carry. An EMIT of one ASCII character (= 8 bits) with $\delta = 0$ costs $8 + 1 = 9$. Ten characters costs $80 + 1 = 81$. You cannot emit more for the same price as less.

For **everything outside that positive-cost class**, including ADD, LOAD, STORE, JUMP, PNEW, PSPLIT, PMERGE, MORPH, COMPOSE, and the rest, the machine charges exactly δ, which can be 0. Here is why these are allowed to be free.

PNEW creates a module. PSPLIT splits one module into two. PMERGE merges two modules into one. These are *reversible*: PNEW can be undone by removing the module; PSPLIT and PMERGE undo each other. Landauer's principle says reversible operations carry zero information-erasure cost. So PNEW at $\delta = 0$ is free, not as a

special exemption, but because there is genuinely no information being destroyed.

The same logic applies to MORPH, COMPOSE, and the other graph-building operations. Building the morphism chain is the organizational work. The irreversible cost happens when you *assert* the chain via MORPH_ASSERT — that is the moment of commitment, and that costs $S(\delta) \geq 1$. You can build for free. You cannot certify for free.

μ grows whenever `instruction_cost` is positive. The certification/revelation class is forced to be positive even when its encoded δ is 0. Other instructions can remain free by carrying $\delta = 0$.

Here is the complete cost schedule:

Instruction class	Cost = `instruction_cost`
Ordinary compute (ADD, LOAD, JUMP, PNEW, MORPH, ...)	δ (can be 0)
Certification setters (CERTIFY, MORPH_ASSERT, LJOIN)	$S(\delta) = \delta + 1$
EMIT (emit payload as certified trace event)	$\lvert\text{payload}\rvert_{\text{bits}} + S(\delta)$
REVEAL (disclose bits from a module)	$\text{bits} + S(\delta)$
READ_PORT (read bits from input port)	$\text{bits} + S(\delta)$
LASSERT (certify a logical formula)	$\mathit{flen} \times 8 + S(\delta)$

The $\lvert\text{payload}\rvert_{\text{bits}}$ notation means "the number of bits in the payload." A one-byte ASCII character contains 8 bits. A ten-byte string contains 80 bits.

For LASSERT: *flen* is the number of formula units in the encoded formula. Each unit is one byte (8 bits) in the binary encoding used by the machine. This is a design choice in the binary format: the formula header declares how many bytes follow, and the machine charges 8 bits per byte. The multiplication by 8 simply converts byte-count to bit-count, matching the information content you're claiming to have certified.

The proof that μ is always exactly $\mu_{\text{before}} +$ `instruction_cost` for each step:
Theorem 6.1 (μ-Conservation)**.** *For any state s and instruction i:* $(\texttt{vm\_apply}\ s\ i).\mu = s.\mu + \texttt{instruction\_cost}\ i$.

This is not a monotonicity claim. It is an equality. μ goes up by exactly the cost of the instruction, always, for every instruction, in every case including error cases. There is no backdoor.

That equality should not be misunderstood. It means the ledger is exact about spend. It does not, by itself, mean every paid unit corresponds to deep semantic gain. In this machine, semantic gain is stricter than spend: it requires a narrowing witness in addition to the ledger entry.

6.2 Why mu is the unique canonical cost measure

Here is the stronger claim, which took me a while to prove: μ is not just A valid cost measure. It is THE cost measure. Any other cost functional that starts at zero and is instruction-consistent — meaning it increases by exactly `instruction_cost` i at each step — must equal μ on every reachable state. Not "isomorphic to" in some abstract sense. Literally equal to, value by value, on every state the machine can actually reach.

Theorem 6.2 (μ-Initiality)**.** *Let M be any function on machine states satisfying: (a) $M($`init_state`$) = 0$ (zero on the canonical initial state), and (b) $M($`vm_apply` $s\ i) = M(s) +$ `instruction_cost` i for all reachable states (tracks costs exactly). Then $M = \mu$ on all reachable states. (Note: monotonicity $M(s') \geq M(s)$ is a consequence of (b), since* `instruction_cost` *returns a natural number which is always ≥ 0. You don't need to state it separately.)*

In plain language: for this machine with this instruction cost assignment, there is no alternative accounting. If you define costs the way the machine does — S(δ) for cert-setters, |payload| + S(δ) for EMIT, and so on — then μ is the only functional that starts at zero and tracks those costs exactly. What was chosen was the cost assignment itself: the decision that CERTIFY costs S(δ) and ADD costs δ. The initiality theorem says that given that assignment, μ is the unique canonical measure. You cannot swap in a different count that agrees on the cost schedule and gets a different total. That freedom is closed.

6.3 The LASSERT cost formula

The most important cost formula in the machine is for LASSERT, the opcode that certifies a logical formula:

$$\Delta\mu_{\text{LASSERT}} = \mathit{flen} \times 8 + S(\mathit{mu\_delta})$$

flen is the encoded formula-unit count, read from the formula's own header in memory. Each unit contributes eight concrete bits, and the per-step certification charge is always at least 1.

The machine reads how many encoded formula units are actually present, charges eight bits for each unit, and adds at least 1 for the certification step. A longer formula costs more. A two-unit formula costs at least 16 + 1 = 17. The programmer does not get to smuggle a larger formula through a smaller bit count.

This is forced, not chosen. You cannot certify a 1000-bit formula by paying for a

10-bit one; the cost counts the formula you actually present. The proof that this is the unique minimum cost satisfying that constraint is in `MuCostDerivation.v` (`cost_necessity` + `cost_uniqueness`).

What this buys you: the formula pays for presenting and checking a constraint package. It does not, by itself, guarantee the constraint is meaningful. You could present a formula that is always true, a tautology, and the ledger would still charge you. The machine closes this separately via the dual-witness requirement: see the LASSERT opcode description in Section 9.

The natural attack, and why it doesn't work. The programmer declares *flen* in the instruction encoding. What stops them from writing *flen* $= 0$ for a huge formula and paying only $S(0) = 1$?

The hardware catches it. When LASSERT runs, the machine reads the formula's actual encoded-unit count from the formula's own header in memory and compares it to the *flen* in the instruction. If they don't match, the machine traps: the program counter jumps to `LASSERT_TRAP_PC` = 3840 = 0xF00 (a fixed trap address), the error flag is set, and the check does not succeed. This trap-to-fixed-address behavior is unique to LASSERT. Every other error in the machine simply sets `vm_err` = true and continues advancing the program counter normally (PC+1). LASSERT is special because a mislength formula is a serious integrity violation, not a recoverable error. The μ cost is still charged. That detail matters. Failed checks are not free. This is proven in `StateSpaceCounting.v` (`lassert_honest_cost` and `lassert_honest_mu_cost`): if LASSERT completes without trapping, the declared count equals the in-memory count, and the μ charge uses the real formula bits.

7 No Free Insight: the first conservation law

7.1 The abstract versions

Theorem 7.1 (Universal No Free Certification)**.** *Let $\mathcal{C}$ be any certification system with:*

- *a state type;*
- *an instruction type;*
- *a step function;*
- *a cost function;*
- *a certification predicate.*

If every single-step transition from uncertified to certified has cost ≥ 1, then any trace that starts uncertified and ends certified has total trace cost ≥ 1.

This theorem is substrate-independent. It does not know about registers, memory, partitions, morphisms, CHSH trials, or the Thiele ISA. It says: once a model has a real certification predicate and the one-step certification transition is priced, trace-level

non-free certification follows by induction. If that one-step premise fails, the model permits free certification. If the model has no certification predicate, it cannot state structural entitlement internally.

Theorem 7.2 (Thiele No Free Insight)**.** *For the Thiele Machine specifically, cert-channel activation and certified predicate strengthening require structure-addition events, and those events are in the positive-cost certification/revelation class.*

This is the machine-level version. `AbstractNoFI.v` proves the two concrete cert channels cannot be activated for free. `NoFreeInsight.v` adds the predicate-strengthening layer: once a stronger accepted predicate is tied to `has_supra_cert`, the run must contain a structure-addition event.

7.2 The specific versions

For the Thiele Machine concretely, there are several specific versions. The two certification channels I mentioned in Section 5 are the formula-certification register (`csr_cert_addr`) and the top-level flag (`vm_certified`). Both are defined precisely in Section 8.

Theorem 7.3 (No Free Certification, single step)**.** *If the formula-certification register goes from absent to present in a single step, the instruction that caused it cost* ≥ 1*.*

Theorem 7.4 (No Free Certification, trace level)**.** *If the formula-certification register was absent at the start of a trace and is present at the end, then the trace paid at least 1 in* μ*.*

Theorem 7.5 (No Free Certification via CERTIFY)**.** *If the top-level certified flag goes from false to true in a single step, the instruction that caused it cost* ≥ 1*.*

Theorem 7.6 (Both channels)**.** *For ANY transition of either certification channel from absent to present, whether the formula register or the certified flag,* μ *grows by at least 1.*

Theorem 7.6 is the master NoFI result. It covers both ways to get certified. There is no third way.

7.3 What this means in practice

Run any program on the Thiele Machine. Watch the μ counter. If the program ends with `vm_certified = true` or with `csr_cert_addr` nonzero, then somewhere in the execution, μ went up by at least 1. You can audit the trace and find exactly where. There is no program that can reach a certified state from scratch without paying for it.

This is what I mean by unforgeable. The μ ledger is the receipt. If you are certified, the receipt exists.

8 The formal machine state

Definition 8.1 (Thiele Machine State)**.** A Thiele Machine state s is a record with 12 fields:

- s.`vm_graph`: the partition graph. It contains module records, morphism records, and fresh-ID counters.

- *s*.vm_csrs: control-status registers. In hardware design, there are two kinds of registers. Data registers (like *s*.vm_regs) hold computation values. Control-status registers hold machine configuration and status. This record has four fields:
  - csr_cert_addr: 0 means nothing has been asserted yet. Set to the ASCII checksum of the property string when MORPH_ASSERT succeeds (nonzero for any non-empty property string). This is not a memory address — it is a checksum fingerprint of the property that was asserted. Having a nonzero value here means a structural property was certified and paid for.
  - csr_status: general status code. Informational; not currently used to gate opcodes.
  - csr_err: error code. Nonzero means an instruction failed (bad argument, missing morphism, locality violation, etc.).
  - csr_heap_base: base address for HEAP_LOAD and HEAP_STORE. Heap-relative memory access adds this base to the register-specified offset.
- *s*.vm_regs: a list of register values. The kernel fixes REG_COUNT = 32; register reads and writes wrap indices modulo 32.
- *s*.vm_mem: a list of data-memory words. The kernel fixes MEM_SIZE = 65536; memory reads and writes wrap indices modulo 65536.
- *s*.vm_pc $\in \mathbb{N}$: the program counter.
- *s*.vm_mu $\in \mathbb{N}$: the structural cost ledger. Starts at 0. Never decreases.
- *s*.vm_mu_tensor: a 4×4 array of cost values (16 entries), indexed by direction pairs (i, j). TENSOR_SET writes an entry, TENSOR_GET reads one. REVEAL records disclosure costs at the direction index encoded in the instruction, so the machine can track *where* a cost was paid across directions, not just how much. In the kernel model these are natural numbers; in the hardware, 32-bit words. If you don't use the tensor tracking, ignore all 16 entries.
- *s*.vm_err $\in \{\text{false}, \text{true}\}$: the error flag. Set when an instruction fails.
- *s*.vm_logic_acc: a natural number serving two roles. First: the running accumulator for the Logic Engine during LASSERT — the on-chip unit that reads formula words and certificate bytes and tracks validation state. Second: a key for high-value operations. When vm_logic_acc equals the constant 0xCAFEEACE (hexadecimal; 3,405,685,454 in decimal), certain high-value opcodes unlock. This field exists so the Coq kernel and the physical hardware agree on its value after every step.
- *s*.vm_mstatus: machine-status word, currently either 0 or 1. 0 means Turing mode: the machine is executing ordinary computation, the partition graph and morphism map are idle. 1 means Thiele mode: structural state is live. In the current kernel, this field is a status indicator that records which operational

context the machine is in — it does not gate the structural opcodes at the step level; the opcodes are always available. The field exists so the hardware and the Coq model agree on machine status across the bridge proofs.

- s.vm_witness: the CHSH witness counters. This is a record of 8 natural-number counters, one for each combination of (measurement setting pair) $\times$ (same/different outcome). The four setting pairs are $(a, b) \in \{(0,0), (0,1), (1,0), (1,1)\}$, using the standard Bell test convention where a is Alice's setting and b is Bob's setting (as in Section 14). For each pair, there are two counters: same-outcome (wc_same_ab) and different-outcome (wc_diff_ab). Eight counters total: wc_same_00, wc_diff_00, wc_same_01, wc_diff_01, wc_same_10, wc_diff_10, wc_same_11, wc_diff_11. Each CHSH_TRIAL instruction increments exactly one counter. (Note: the CHSH_TRIAL opcode and the Coq code name the setting parameters x, y and the outcome parameters a, b — the reverse of this standard Bell test convention. The counter subscripts 00, 01, 10, 11 are numeric and unambiguous regardless of which letter names the settings.) The CHSH score S is computed from the aggregate sums of these counters. For each setting pair (a, b), define $N_{ab} = \texttt{wc\_same}_{ab} + \texttt{wc\_diff}_{ab}$ (total trials with that setting) and $E_{ab} = (\texttt{wc\_same}_{ab} - \texttt{wc\_diff}_{ab})/N_{ab}$ (the correlation: fraction same minus fraction different, ranging from -1 to $+1$). Then $S = E_{00} + E_{01} + E_{10} - E_{11}$. If no trials have been run for a setting ($N_{ab} = 0$), the correlator for that setting defaults to 0 by convention. Classically $|S| \leq 2$; the NPA algebraic model allows $|S| \leq 2\sqrt{2}$ (see Section 14). The concrete Coq definition is chsh_stat_from_wc in CHSHStatisticalBridge.v.
- s.vm_certified $\in$ {false, true}: the top-level certification flag. Set by the CERTIFY opcode.

The Coq kernel (the mathematical ground truth) and the physical hardware are intentionally not the same shape at every bit. The Coq kernel stores register and memory values as natural numbers, but truncates writes through 64-bit word helpers to keep the math clean. The hardware layer (explained in Section 13) is finite: 32-bit data registers, 65536 memory words, bounded tables. The bridge proofs in the hardware section state exactly where those finite hardware representations match the kernel step.

Definition 8.2 (Partition Graph)**.** The partition graph s.vm_graph is a record:

- pg_next_id: the next fresh module identifier.
- pg_modules: a list of module-ID/module-record pairs. Each ModuleState holds region data, axiom data, and the module's μ-tensor.
- pg_next_morph_id: the next fresh morphism identifier.
- pg_morphisms: a list of morphism-ID/morphism-record pairs. Each MorphismState holds a source module, target module, coupling data, and

an identity flag.

Remark 8.1. There is no separate `vm_heap`, `vm_output`, or `vm_imem` field in the Coq kernel's `VMState`. Heap operations are heap-relative accesses into `vm_mem`. Output and instruction transport live at the runner, trace, or RTL harness layer, not as kernel state fields. This distinction matters because NoFI is a theorem about the state record above.

Remark 8.2. The partition graph has two distinct layers. The *module layer* records how the state is decomposed into named pieces. The *morphism layer* records typed categorical relationships between those pieces. These layers are independent. Two machine states can agree on every module while differing in their morphism maps. This independence is what makes the Categorical Separation Theorem possible (Section 11).

9 The instruction set: all 46 opcodes

The Thiele Machine has 46 opcodes. Every one carries an explicit `mu_delta` parameter. Cost is part of the instruction, not an annotation.

9.1 Group 1: Partition operations (4 opcodes)

These opcodes modify the partition graph. They are how the machine builds structural knowledge.

PNEW(R, δ): allocate a new module.
Creates a fresh module and charges δ to μ. The instruction carries a region list R describing which memory addresses belong to this module. The current hardware-aligned semantics normalize this to a canonical form: they only keep the size (how many addresses) and store it as the range `0, 1, ..., sz-1`. So the checked step preserves the module size, not arbitrary input geometry. The bounded RTL has its own partition-table and active-module machinery for locality enforcement, but `PNEW` itself is the graph allocation step.

Why does this exist? Because structural knowledge needs a home. Creating a module is how you tell the machine "this region of computation is a named unit I'm tracking." The cost δ is programmer-declared and can be 0; the machine does not force you to pay for allocation itself. If building this partition cost something because you had to figure out the right boundary, run a search, or derive the structure, you record that here. If it was bookkeeping, $\delta = 0$ is fine.

PSPLIT(m, L, R, δ): split one module into two.
Splits module m and charges δ to μ. The constructor still carries left and right region payloads, but the current hardware-aligned step ignores those payloads and calls `graph_hw_psplit`: the left side gets half the module size, and the right side gets the remainder. This is the refinement operation in the finite hardware model. If discovering the split deserves a cost, encode a positive δ; the canonical reversible-accounting policy keeps it at 0.

PMERGE(m_1, m_2, δ): merge two modules into one.
Merges modules m_1 and m_2 and charges δ. The hardware-aligned step concatenates the module sizes through `graph_hw_pmerge`; module IDs wrap modulo 64. It does not perform a rich invariant-compatibility check in this step. Like PNEW and PSPLIT, the machine does not enforce a minimum cost here. The mandatory cost comes later if you try to certify a claim about the structure.

PDISCOVER(m, *evidence*, δ): query partition structure.
The evidence payload is carried by the instruction, but the hardware-aligned relation does not attach axioms or mutate the graph. It advances the PC and charges δ. This is a placeholder for discovery accounting, not a register query in the current kernel step.

9.2 Group 2: Logic and certification (3 opcodes)

These opcodes interact with the Logic Engine, the on-chip finite-state machine that verifies formulas and certificates.

LASSERT(*freg*, *creg*, *kind*, *flen*, *cost*): assert and certify a formula.
Reads a formula whose declared encoded-word count is *flen* from memory starting at the address in register *freg*. The *kind* boolean flag selects which certificate path the Logic Engine should follow. "SAT" (satisfiable) means: I'm claiming this formula has at least one satisfying assignment — some combination of variable values that makes it true. "UNSAT" (unsatisfiable) means: I'm claiming it has no satisfying assignment — it's always false. In the current on-chip semantics, *kind* = *true* is the SAT path (dual-witness: one satisfying assignment and one falsifying assignment). *kind* = *false* is the UNSAT path: it runs a unit-propagation conflict checker — scanning the formula for empty clauses and direct unit-clause negation conflicts. This is sound but not complete: it catches unit-propagation-refutable UNSAT formulas, not all unsatisfiable formulas. No external certificate is required; the formula itself is the evidence. This matches the RTL phase-3/4 FSM exactly. The kernel and hardware both validate that the declared count matches the in-memory formula header before the check is allowed to succeed. A mismatch traps and sets the error flag. Reads a certificate block from memory starting at the address in register *creg*. In the tightened architecture, that block contains two witnesses: one satisfying assignment and one falsifying assignment. The on-chip Logic Engine FSM processes the formula and both witnesses in-place without external solver calls. Validation succeeds only if the declared count is honest, the first witness satisfies the formula, and the second witness falsifies it. That means the formula is not a tautology and really excludes at least one state. LASSERT itself is a checked validation step; certification channels are activated by the concrete state-changing paths such as CERTIFY and MORPH_ASSERT, while the abstract cert-address theorem treats LASSERT as part of the certification/revelation class. Charges $\mathit{flen} \times 8 + S(\mathit{cost})$ to μ, even when the check fails.

This is the most important logic opcode in the machine. It is where a constraint package is checked before any later certification step can lean on it. You cannot fake the pricing

anymore by declaring a tiny *flen* for a large in-memory formula: if the header and the instruction disagree, the machine traps. You also cannot fake semantic narrowing with a tautology: the dual-witness requirement closes the tautology-inflation hole because there is no falsifying witness to show the formula excluded any state. The cost formula is still a spend ledger. The honest-length check and the non-triviality witness are what tie that spend to the real checked object.

LJOIN(*c1reg*, *c2reg*, δ): join two certificates.
Combines two memory-resident certificates into a single composite certificate. The addresses of the input certificates are in *c1reg* and *c2reg*. Charges $S(\delta) \geq 1$. This supports constructing complex multi-part proofs from simpler pieces.

MDLACC(*m*, δ): MDL cost accounting.
Charges δ as a module-discovery/model-description cost for module m. It does not read a register, and it does not mutate the graph.

Suppose you have two models that both fit the same data. One model is a ten-thousand-parameter neural network. The other is a five-line formula. Both fit the training set. Which one are you buying? The minimum-description-length principle says: the total bill is the model description itself plus the cost of encoding the data once you have the model. The description length of a model is how many bits you need to write it down — a simple formula takes very few bits; a massive network takes many. The short formula costs more to encode data with, but costs almost nothing to describe. The massive network achieves perfect in-sample compression, but its own description is enormous. The correct choice minimizes the sum of (bits to describe the model) + (bits to encode the data given the model).

Translated to the Thiele Machine vocabulary: a module structure that you built has a description-length cost. Not because someone told you it does, but because constructing a specific partition of state space may require information that should be billed. MDLACC is the opcode that lets a program explicitly enter that description cost into the ledger after the structure has been built. It performs no graph change. Its sole effect is to advance the program counter and charge the encoded δ. The connection to MDL is not a citation to a 1978 paper; it is the claim that the machine's natural μ-accounting can track what information-theoretic compression theory says a model costs, when the program chooses to enter that cost.

9.3 Group 3: Compute and data transfer (14 opcodes)

Standard arithmetic and data movement. These are the operations you find in every ISA. They mostly carry $\delta = 0$ because they do not introduce new structural knowledge. They just compute.

Why does the ISA have all these? Because the Thiele Machine is a complete computing machine, not just a structural accounting system. Programs need to compute interme-

Opcode	What it does	Typical cost
`LOAD_IMM`(r, v, δ)	Write immediate value v into register r	0
`XFER`(r_{dst}, r_{src}, δ)	Copy register r_{src} to r_{dst}	0
`ADD`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a + r_b$ (modular 64-bit)	0
`SUB`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a - r_b$ (modular 64-bit)	0
`MUL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a \times r_b$	0
`AND`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ AND r_b (bitwise)	0
`OR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ OR r_b (bitwise)	0
`SHL`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted left by r_b bits	0
`SHR`(r_d, r_a, r_b, δ)	$r_d \leftarrow r_a$ shifted right by r_b bits	0
`LUI`(r_d, v, δ)	Load upper immediate: $r_d \leftarrow v \ll 8$	0
`XOR_LOAD`(r_d, *addr*, δ)	Load from absolute memory address *addr* into r_d	0
`XOR_ADD`(r_d, r_s, δ)	$r_d \leftarrow r_d$ XOR r_s (XOR accumulate)	0
`XOR_SWAP`(r_a, r_b, δ)	Swap registers r_a and r_b (reversible, direct two-write exchange)	0 by convention
`XOR_RANK`(r_d, r_s, δ)	$r_d \leftarrow \text{popcount}(r_s)$ (Hamming weight)	0

diate values, manage data, and control flow. The compute group provides all of that. The key property: none of these opcodes touch the certification channels. They cannot create certified structural knowledge.

XOR_SWAP is a reversible register swap. The kernel implements it as a direct two-write exchange (not the XOR-trick sequence $a \oplus= b; b \oplus= a; a \oplus= b$, despite the name). It carries $\delta = 0$ by convention because a reversible swap does not increase information entropy.

9.4 Group 4: Memory and control flow (8 opcodes)

Memory access and program flow control.

Opcode	What it does	Note
`LOAD`(r_d, r_a, δ)	Load `vm_mem`$[r_a]$ into r_d	Kernel register-indirect load
`STORE`(r_a, r_s, δ)	Store register r_s into `vm_mem`$[r_a]$	Kernel register-indirect store
`JUMP`(addr, δ)	Set PC to addr unconditionally	none
`JNEZ`(r, addr, δ)	Jump to addr if $r \neq 0$	none
`CALL`(addr, δ)	Push return address via stack pointer r31, jump to addr	Qed coverage
`RET`(δ)	Pop return address via stack pointer r31, jump there	Qed coverage
`HALT`(δ)	Advances PC, charges δ. Stops when PC exceeds program length. HALT in the middle does not stop execution.	runner-level
`CHECKPOINT`(ℓ, δ)	Accept label ℓ, advance PC	label outside kernel

At the kernel level, LOAD and STORE are register-indirect memory operations over `vm_mem`. At the RTL/cosimulation layer, the partition wall adds active-module locality checks. `INIT_PT` and `INIT_ACTIVE_MODULE` are harness setup commands for that finite hardware table, not VM opcodes and not fields of `VMState`. This is the honest split: the abstract ISA defines memory semantics; the bounded hardware layer enforces locality through its own table state.

9.5 Group 5: Revelation, I/O, heap, and tensor (7 opcodes)

This group mixes two fundamentally different kinds of operations. EMIT and READ_PORT are in the *certification/revelation cost class* — they cost ≥ 1 unconditionally and are subject to the same mandatory-floor rule as CERTIFY and MORPH_ASSERT. WRITE_PORT, HEAP_LOAD, and HEAP_STORE are ordinary I/O and storage, no mandatory floor. TENSOR_SET and TENSOR_GET manage per-module cost tensors. The shared table is a historical artifact; the cost semantics are distinct.

Opcode	What it does	Cost
`READ_PORT`(r, c, v, *bits*, δ)	Write value v into register r. The value v is embedded in the instruction encoding at assembly time (the kernel step is deterministic: it writes the declared v, not a runtime port read). Channel c and the *bits* count are used by the runner/environment layer for I/O orchestration and cost accountability.	$bits + S(\delta) \geq 1$
`WRITE_PORT`(c, r_s, δ)	Send register r_s to channel c outside kernel state	δ
`EMIT`(m, *payload*, δ)	Emit payload as a certified trace event	$\lvert payload\rvert_{\text{bits}} + S(\delta) \geq 1$
`HEAP_LOAD`(r_d, r_a, δ)	Load from `csr_heap_base + regs[`r_a`]` into r_d	δ
`HEAP_STORE`(r_a, r_s, δ)	Store r_s to `csr_heap_base + regs[`r_a`]`	δ
`TENSOR_SET`(m, i, j, v, δ)	Set per-module tensor entry (i, j)	δ
`TENSOR_GET`(r_d, m, i, j, δ)	Read per-module tensor entry (i, j) into r_d	δ

EMIT and READ_PORT are in the certification/revelation cost class, but their cost goes beyond the simple $S(\delta)$ floor. EMIT charges $|\text{payload}|_{\text{bits}} + S(\delta)$: the machine unfolds the emitted payload into concrete bits and charges for those bits directly, then adds at least 1 for the certification step. A one-byte ASCII payload has 8 payload bits, so with $\delta = 0$ it costs 9. A ten-byte ASCII payload has 80 payload bits, so with $\delta = 0$ it costs 81. The programmer cannot emit more bits for the same price as fewer bits. READ_PORT charges $bits + S(\delta)$: the number of bits being read from the port, plus at least 1. More information in means more μ paid. The floor is still there; zero-cost certification remains impossible, and the cost tracks the actual bit content.

TENSOR_SET and TENSOR_GET manage per-module metric tensor entries, storing the 4×4 geometric weight matrix used by the morphism algebra. The kernel tensor is a flattened 4×4 vector. The hardware has dedicated tensor state, and the bisimulation proofs cover both TENSOR opcodes unconditionally in `GraphReconstructionBridge.v` and `RTLGapRegistry.v`. Bad tensor indices latch error in both semantics; good indices agree step-for-step.

9.6 Group 6: Witness and certification (3 opcodes)

These opcodes accumulate binary game outcome statistics into hardware witness counters and manage the top-level certification flag.

CHSH_TRIAL(x, y, a, b, δ): record one CHSH trial.
Records one trial of the CHSH game. $x \in \{0, 1\}$ is Alice's measurement setting. $y \in \{0, 1\}$ is Bob's measurement setting. $a \in \{0, 1\}$ is Alice's outcome. $b \in \{0, 1\}$ is Bob's outcome. The Coq constructor order is settings first (x, y), then outcomes (a,

b). The instruction increments exactly one of the 8 WitnessCount counters: if $a = b$ (same outcome), it increments `wc_same_xy`; if $a \neq b$ (different outcome), it increments `wc_diff_xy`. There are 4 setting pairs $\times$ 2 same/different buckets = 8 counters. The CHSH score S is computed later from these counters. Charges δ to μ.

This is a Tier 0 operation: it records data but does not activate any certification channel. It is provably free in the sense that CHSH_TRIAL does not and cannot trigger a cert-insight event (proven in `WitnessInsightGeneral.v`, `chsh_trial_is_tier0`). One validation: if any of x, y, a, b is not in {0, 1} — for example, passing x=5 — the machine fires a bad-bits error step: the error flag latches, no counter is incremented, but the cost is still charged. The machine does not silently count invalid inputs as valid trials.

CERTIFY(δ): top-level proof-carrying certification.
Sets `vm_certified = true`, charges $S(\delta) \geq 1$ to μ, advances PC. This is the simplest cert-channel activation: you are saying "I, the program, certify that this execution has reached an attested state." The flag is set. The cost is charged. There is no way to set `vm_certified = true` without this opcode, and this opcode always charges ≥ 1.

REVEAL(*m*, *bits*, *cert*, δ): disclose a value.
Reveals *bits* bits from module *m* with certificate string *cert*. A bit-commitment protocol is a two-phase scheme: first you commit to a value (seal it in an envelope), later you reveal it. The reveal step proves you didn't change your answer after seeing the other side's response. REVEAL is the disclosure half of that scheme. REVEAL charges bits $+ S(\delta)$: the number of bits being disclosed, plus at least 1. The cost is also recorded in the μ-tensor at the direction index encoded in the instruction, marking *where* in the morphism geometry the disclosure cost lands. Revealing 0 bits still costs at least 1. Revealing 32 bits costs at least 33. You cannot disclose more for the same price as less. This is what makes the revelation requirement concrete: obtaining attested cross-correlations from binary game trials requires an explicit disclosure instruction, and that disclosure is priced by how much you reveal.

MORPH_ASSERT (see Group 7): it activates the cert channel and belongs conceptually here too, but is listed in the morphism group because it operates on a morphism ID.

9.7 Group 7: Categorical morphism operations (7 opcodes)

These are the opcodes that place a program outside the classical fragment (`is_classical_opcode` returns false for all of them). A Turing Machine can simulate programs using these opcodes by encoding the full Thiele state on its tape, but only by also implementing the μ-ledger cost semantics — any simulation that omits that accounting loses the structural entitlement guarantees.

A *morphism* is a record saying: "module A is related to module B in this specific,

named way." Not just "A and B exist." Not just "A and B are connected by an edge." A morphism has a source module ID, a target module ID, coupling data (a list of (source-cell, target-cell) pairs plus a label string), and an is-identity flag.

The coupling data is actual relational content. MORPH creates a morphism with an empty coupling list initially. COMPOSE computes the relational composition of two morphisms' coupling lists: pair (a, c) is included if and only if there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct in `CategoryBridge.v`. MORPH_TENSOR concatenates two coupling lists. The coupling is not a programmer tag. It is data the machine builds and proves properties about. You can read back the number of coupling pairs via MORPH_GET (selector 2).

Why does this matter? Because two programs can have identical classical state (same registers, same memory, same μ) while having completely different morphism maps. A classical machine cannot distinguish them. The Thiele Machine can, via a single morphism probe. This is the Categorical Separation Theorem (Section 11).

Opcode	Effect	Cost
`MORPH`(r_d, s, t, c, δ)	Create morphism $s \to t$, write ID into r_d. Empty coupling. (c carried but unused.)	δ
`COMPOSE`(r_d, m_1, m_2, δ)	Compose $m_1 : A \to B$ with $m_2 : B \to C$ (requires target of m_1 = source of m_2). Coupling = relational composition.	δ
`MORPH_ID`(r_d, m, δ)	Identity morphism on module m.	δ
`MORPH_DELETE`(id, δ)	Delete morphism id. Error if not found.	δ
`MORPH_ASSERT`(id, $prop$, $cert$, δ)	Assert morphism id exists with non-empty property $prop$. Sets `csr_cert_addr` = checksum($prop$) on success. Always charges $S(\delta) \geq 1$.	$S(\delta)$
`MORPH_TENSOR`(r_d, m_1, m_2, δ)	Tensor $m_1 \otimes m_2$: requires disjoint source/target regions. New morphism from union-source to union-target. Coupling = concatenation.	δ
`MORPH_GET`(r, id, sel, δ)	Read field (0=source, 1=target, 2=#coupling-pairs, 3=is-identity) of morphism id into r.	δ

MORPH_ASSERT deserves special attention. It charges $S(\delta) \geq 1$ regardless of whether the assertion succeeds. This is the sharpest instance of No Free Insight: the *attempt to claim structural knowledge costs something*, even when the claim is false. A failed assertion is not free. A machine without a structural ledger has no way to express this constraint. In the Thiele Machine it is enforced by the step semantics and proven in `AbstractNoFI.v`.

9.8 Why 46 opcodes?

The number 46 comes from what the machine needs to do:

- Compute arithmetic and data: 14 ops (LOAD_IMM, XFER, ADD, SUB, MUL, AND, OR, SHL, SHR, LUI, XOR_LOAD, XOR_ADD, XOR_SWAP, XOR_RANK)
- Memory and control flow: 8 ops (LOAD, STORE, JUMP, JNEZ, CALL, RET, HALT, CHECKPOINT)
- I/O, heap, and tensor: 7 ops (READ_PORT, WRITE_PORT, EMIT, HEAP_LOAD, HEAP_STORE, TENSOR_SET, TENSOR_GET)
- Partition structure: 4 ops (PNEW, PSPLIT, PMERGE, PDISCOVER)
- Logic and certification: 3 ops (LASSERT, LJOIN, MDLACC)
- Witness and certification flags: 3 ops (CHSH_TRIAL, CERTIFY, REVEAL)
- Categorical morphisms: 7 ops (MORPH, COMPOSE, MORPH_ID, MORPH_DELETE, MORPH_ASSERT, MORPH_TENSOR, MORPH_GET)

$14 + 8 + 7 + 4 + 3 + 3 + 7 = 46$.

Every opcode exists for a reason. None are redundant. If you removed CERTIFY, you could not activate `vm_certified`. If you removed MORPH, you could not build morphism relationships. If you removed CHSH_TRIAL, you could not track 2-player binary game statistics. The ISA is designed, not grown.

10 What a categorical morphism is: the category theory, from scratch

Category theory from scratch. I didn't know what it was when I started this project.

10.1 The idea of a category

A category is a collection of two things: objects and arrows between them.

The objects can be anything. In abstract math, they might be sets, or vector spaces, or topological spaces. In the Thiele Machine, the objects are modules (partitions of state).

The arrows are also called morphisms. An arrow goes from one object to another: $f : A \to B$ means "there is an arrow f from object A to object B." The arrow does not have to be a function. In the Thiele Machine, a morphism just says "there is a relationship of this type between these two modules."

Two laws must hold:

1. **Composition**: If $f : A \to B$ and $g : B \to C$, there must exist a composite $g \circ f : A \to C$. This is what COMPOSE implements.
2. **Identity**: For every object A, there must exist an identity arrow $\mathrm{id}_A : A \to A$. This is what MORPH_ID implements.

And composition must be associative: $(h \circ g) \circ f = h \circ (g \circ f)$.

Why composition? Because you want to chain relationships. If A relates to B and B relates to C, you want to say A therefore relates to C, transitively, without having to name every possible A-to-C path individually. Without composition, the structure collapses into a flat list of unconnected pairs. It's entirely useless for multi-hop reasoning.

Why identity? Because every object needs a minimal self-relationship. An isolated module with no connections to anything else still needs to be expressible in the framework. MORPH_ID is how you say "module A relates to itself with no coupling content." Without it, isolated objects fall outside the framework entirely, and any module with zero external relationships becomes unrepresentable.

Why associativity? Because the parenthesization of a chain should not matter. Whether you write $(h \circ g) \circ f$ or $h \circ (g \circ f)$, both mean the same three-hop path $A \to B \to C \to D$. If associativity failed, two programs that build the same chain in different groupings would produce different morphisms. That is a bug: the chain is the thing, not how you assembled it.

These laws are proven in Coq in `CategoryLaws.v`. Concretely: morphisms carry coupling data, a list of (source-cell, target-cell) pairs plus a label. A "cell" is a natural number indexing into a module's memory region. A coupling pair (a, c) says: cell a in the source module is related to cell c in the target module. The coupling is the *relational content* of the morphism — the actual evidence of which parts of one module correspond to which parts of another. COMPOSE produces a new morphism whose coupling pairs are the relational composition of the two inputs: (a, c) is included when there exists an intermediate cell b such that (a, b) is in the first coupling and (b, c) is in the second. This is the standard relational composition (like matrix multiplication but for sets of pairs). This is proven in `CategoryBridge.v` (`graph_compose_morphisms_coupling`). The Thiele Machine's categorical layer is not just called a category; it satisfies the actual axioms on the actual coupling data.

10.2 Why modules as objects

In the Thiele Machine, the objects of the category are the modules of the partition graph. A module is a named piece of the computation: it owns a region of memory or state, it may carry certified axioms about that region, and it contributes a local μ cost.

A morphism between two modules says: "there is a named relationship between module A and module B, with explicit coupling data." The coupling data is a list of (source-cell, target-cell) pairs. COMPOSE computes the relational composition of two couplings: pair (a, c) is in the composed coupling exactly when there exists b such that (a, b) is in the first coupling and (b, c) is in the second. This is proven correct. The machine tracks it. Via MORPH_GET (selector 2), you can read back the number of

coupling pairs in any morphism.

What the coupling pairs represent is up to the programmer. They could encode:

- Module B refines module A (which cells in A map to which cells in B)
- Module A entangles with module B (which cell-pairs are correlated)
- Module A simulates module B (which states correspond)

The machine enforces the relational composition correctly. The interpretation is yours.

10.3 Why this is not just "extra state"

Here is the question I asked myself for a long time: if you can just add extra registers to a classical machine to store morphism IDs, why is the categorical layer special?

The answer is in the MORPH_ASSERT opcode and in the Categorical Separation Theorem.

Extra registers do not have provenance. You can write anything into a register. You can write 42 into register 5 and claim it is a morphism ID. But the Thiele Machine checks: did a MORPH instruction actually run that created morphism 42 with the source and target you claim? If not, MORPH_ASSERT fails. And MORPH_ASSERT costs $S(\delta) \geq 1$ even if it fails.

Building the morphism chain by calling MORPH, COMPOSE, and MORPH_ID is free when those instructions carry $\delta = 0$. Those ops don't have a mandatory positive floor. What costs is MORPH_ASSERT: the act of asserting that the morphism exists and activating the cert channel. That always costs $S(\delta) \geq 1$. You can build the chain for free, but you cannot certify it for free.

This is why the morphism layer is not "extra state." It is *provenance-bearing state*. The machine knows where it came from. A classical machine cannot fake this at the same cost. That is the Categorical Separation Theorem.

11 Categorical separation: the machine is strictly richer

Theorem 11.1 (Categorical Separation)**.** *There exist states s_1 and s_2 such that:*

- $s_1.\texttt{vm\_regs} = s_2.\texttt{vm\_regs}$ *(identical registers)*
- $s_1.\texttt{vm\_mem} = s_2.\texttt{vm\_mem}$ *(identical memory)*
- $s_1.\texttt{vm\_mu} = s_2.\texttt{vm\_mu}$ *(identical μ)*
- $s_1.\texttt{vm\_err} = s_2.\texttt{vm\_err}$ *(identical error flag)*
- $s_1.\texttt{vm\_pc} = s_2.\texttt{vm\_pc}$ *(identical program counter)*
- $s_1.\texttt{vm\_certified} = s_2.\texttt{vm\_certified}$ *(identical top-level certification flag)*

yet their morphism graphs differ. In the concrete witness, one state has an identity morphism in `pg_morphisms`*; the other has no morphisms.*

Construction. Build s_1 with `pg_morphisms = [(0, id)]`. Build s_2 with `pg_morphisms = []`. Set every classical field the same: registers, memory, μ, PC, error flag, and `vm_certified`. The equality proof is by reflexivity on those fields. The graph distinction is by list constructor mismatch: a one-element morphism list is not the empty list. □

Theorem 11.2 (Shadow Separation)**.** *There exist two states with the same classical shadow but different morphism graphs, and a legitimate graph-preserving probe after which the morphism-level distinction remains.*

The probe used in `ShadowProjection.v` is deliberately boring: `ADD 0 0 0 0`. It is not a morphism operation. That is the point. The theorem is not relying on a failing probe to manufacture a difference. The difference is already in the structural state, and ordinary classical projection cannot see it.

Corollary 11.3 (Classical observer is blind, `ShadowProjection.v`)**.** *No function of (registers, memory, μ, PC, error, certified) can separate all Thiele states. The classical shadow is strictly lossy.*

Corollary 11.4 (Thiele strictly extends classical, `TuringStrictness.v`)**.** *Every program in the classical fragment — programs restricted to* `is_classical_opcode`*, meaning no graph mutations, no cert-channel ops, no witness ops — leaves* `pg_next_id` *unchanged. There exist Thiele programs that change* `pg_next_id` *(via PNEW). Therefore Thiele strictly extends the classical fragment: it can represent and probe states that no program in that fragment can reach. A Universal Turing Machine can simulate the full Thiele state by encoding it on tape; what it cannot do is produce that state without expending the equivalent structural work — the μ-ledger cost is preserved under any faithful simulation.*

The classical machine simulates inside the Thiele Machine faithfully (every classical program runs correctly, no side effects on the structural layer). But the Thiele Machine has programs that escape the classical fragment. This is proven, not claimed.

12 The knowledge receipt: what makes certification unforgeable

Four concrete facts about why the μ ledger is unforgeable.

Act 1: The attempt costs, even when it fails.

Run MORPH_ASSERT(k, "test", "", 0) where morphism k does not exist. What happens:

- `vm_err` becomes true (the assertion failed — morphism k not found).
- `csr_cert_addr` stays 0 — the cert channel is NOT activated on failure.
- μ increases by $S(0) = 1$ regardless (the attempt cost something).

The machine paid for a failed assertion. A machine without a structural ledger has no way to express this. In the Thiele Machine, the cost of claiming knowledge is charged whether or not the claim is true.

Act 2: Build the chain.

Run this sequence:

$$\text{PNEW}(A),\ \text{PNEW}(B),\ \text{PNEW}(C)$$
$$\text{MORPH}(f, A, B),\ \text{MORPH}(g, B, C)$$
$$\text{COMPOSE}(h, f, g)$$

Then MORPH_GET(r_0, h, source) and MORPH_GET(r_1, h, target). The resulting state has $r_0 = A$, $r_1 = C$, and morphism $h : A \to C$ exists in the morphism map. With $\delta = 0$ on each build step, $\mu = 0$. The morphism exists, but no structural cost has been paid yet.

Act 3: Certify.

Run MORPH_ASSERT(h, "composed", "", 0). Morphism h exists and the property string "composed" is non-empty. The assertion succeeds. `csr_cert_addr` goes to the ASCII checksum of "composed" (nonzero). Tier 1 cert channel activated. μ goes up by $S(0) = 1$.

(Why must the property string be non-empty? Because csr_cert_addr stores the ASCII checksum of the property string. If property = "", checksum = 0, and the channel stays at 0 — not activated. Any non-empty string works; "composed" is just an example label.)

The machine now holds a μ-ledger entry: at this point in the trace, a morphism assertion was made and paid for. The receipt is in the ledger. It cannot be removed. It cannot be backdated. μ never goes down.

Act 4: Classical programs cannot forge it.

Program B takes a shortcut: it loads $r_0 = A$ and $r_1 = C$ via LOAD_IMM. Classical state is identical: same registers, same μ (both zero). No morphism h was ever created.

Apply MORPH_DELETE(h): Program A (Act 2) succeeds. Program B fails and sets err.

The μ ledger is not a counter that measures how much work was done. It is a record of which structural commitments were made. Program B did the same amount of "work" (in the μ sense) but did different work. The machine knows the difference.

13 How the machine is built: three layers

The Thiele Machine exists in three forms. They are intended to implement the same semantics. The three layers are tied together by refinement proofs and automated tests.

13.1 Layer 1: The Coq kernel

This is the mathematical ground truth. The Coq kernel defines:

- The VMState record (all 12 fields, as in Section 8)
- The vm_instruction type (all 46 opcodes as Coq inductive constructors)
- The vm_apply function: takes a state and instruction, returns a state. Total. Always returns something.
- All the theorems: NoFI, μ-monotonicity, μ-initiality, categorical separation, Turing strictness, etc.

The Coq kernel operates mostly over natural numbers, but the machine state is not an unbounded fantasy object. It fixes 32 registers and 65536 memory words, and its accessors wrap indices through those sizes. Register and memory writes are truncated by the kernel's 64-bit word helpers. μ itself is a natural number and can grow without a fixed machine-word ceiling in the kernel. The RTL target then instantiates a finite 32-bit datapath and bounded tables.

Zero Admitted means: no step in any theorem proof was asserted without derivation. The machine checked everything. I did not.

13.2 Layer 2: The OCaml extracted runner

Coq can automatically extract executable code from a Coq development. The language it targets here is OCaml — a general-purpose programming language. "Extraction" means: Coq keeps the computational content of the definitions and throws away the proof terms. A function like `vm_apply` (which defines what every opcode does) extracts to an OCaml function that computes the same thing. The extracted result is not a translation of the proofs — the proofs are erased. What remains is the algorithm. The extracted runner executes all 46 opcodes from the Coq semantics, including the morphism operations.

The trust boundary here is named explicitly. `OCamlExtractionBridge.v` proves formal facts about the Coq-side observable `shadow_to_eo (vm_apply s i)`: exact μ accounting, nondecreasing μ, and totality. It also names the external runner boundary. The current file does not contain a real unproved Coq `Hypothesis` called `ocaml_runner_agrees`; the lemma with that name is a reflexive Coq-side witness. The actual cross-language claim, that the built OCaml binary returns the same observable as the Coq semantics, is checked by the parity test suite. I don't count that as a kernel theorem. It is an audited extraction boundary backed by tests.

13.3 Layer 3: The Kami RTL hardware

The Kami framework lets you write hardware specifications in Coq and extract them to Bluespec SystemVerilog. Bluespec SystemVerilog is a high-level hardware description language: you describe what the hardware should compute, including the registers, the rules, and the conditions under which state transitions fire. The Bluespec compiler generates standard Verilog RTL. Verilog RTL is the low-level gate-and-register description that synthesis tools can compile into actual logic gates on an FPGA (a reconfigurable chip you can program after manufacturing) or an ASIC (a custom chip manufactured for a specific design, not reprogrammable). The Thiele Machine hardware is defined in `ThieleCPUCore.v` (Kami module) and extracted through this pipeline.

The hardware has:

- 32 registers, each 32 bits wide (data registers)
- 128-bit instruction encoding (`InstrSz = 128`)
- 65536 words of data memory
- bounded instruction transport and execution state
- 64 partition module slots (`PTableSz = 64`)
- 64 morphism descriptor slots (`MorphTableSz = 64`)
- An on-chip LASSERT FSM for formula verification
- A μ-ALU running in parallel with main execution

All 46 opcodes are implemented in the RTL design: `ThieleCPUCore.v` has hardware logic for all of them. The question is not which opcodes are in hardware, but which opcodes have a completed formal *bisimulation proof*.

A bisimulation proof says: given the same starting state and the same instruction, the hardware and the Coq model produce the same output state. Not "similar" or "equivalent in some abstract sense." Literally: the hardware's answer equals the Coq model's answer, field by field. It is a step-by-step correspondence, verified in Coq.

Formal bisimulation proofs exist for all 46 opcodes (zero Admitted, all Qed). Here is the precise breakdown:

- **30 opcodes are truly unconditional.** These are the `SupportedOpcode` group: `True` in Coq, no conditions whatsoever. They cover all standard compute, load/store, jump, I/O, and certification operations that don't involve morphism tables or formula checking. Verified in `EmbedStep.v` (`embed_step_compute`).
- **6 opcodes require valid instruction arguments.** The hardware and Coq model

AGREE on the error path for invalid arguments, so no semantic disagreement exists — but the success-path proof requires the argument to be in range:

- CALL: stack pointer must be in bounds (`WellFormedSnapshot`), and `pc` < 65536.
- RET: stack pointer must be in bounds (`WellFormedSnapshot`).
- CHSH_TRIAL: all four values x, y, a, b must be in $\{0, 1\}$. Passing 5 as an outcome value triggers the bad-bits error path — on which both hardware and VM agree.
- TENSOR_SET: indices i, j must both be < 4 (valid for a 4×4 tensor).
- TENSOR_GET: indices valid AND the module must exist in the graph.
- LASSERT: the declared formula-unit count *flen* must match the actual in-memory formula header. (Mismatch causes a trap to 0xF00, on which both sides agree; the precondition is needed for the success-path proof.)

Covered in `EmbedStep_WF.v` (CALL, RET, CHSH_TRIAL) and `GraphReconstructionBridge.v` (TENSOR_SET, TENSOR_GET, LASSERT).

- **7 opcodes are unconditional for KamiReachable hardware states.** MORPH_DELETE, MORPH_ASSERT, MORPH_GET, COMPOSE, MORPH_TENSOR, MORPH, and MORPH_ID are proved unconditional for any state reachable from hardware reset via the `KamiReachable` inductive predicate (`HardwareReachability.v`). The invariants morph_table_wf, coupling_wf, sizes_nonzero_bounded hold inductively from reset and are sufficient to close both success and error paths.
- **3 opcodes have a semantic-level disagreement between hardware and VM for certain inputs.** PNEW requires a non-empty region; PSPLIT and PMERGE require existing modules with sufficient size and available table slots. The distinction from the 6 argument-validity group: for PNEW/PSPLIT/PMERGE with invalid inputs, the hardware and abstract VM produce DIFFERENT graph structures (snap_pt_to_graph filters size-0 modules; graph_add_module includes them). No conditions close this gap without changing the semantics.

The project classification (documented in `CloseoutVerification.v`) groups the 30+6 as "36 unconditional" on the grounds that the 6 argument-validity conditions represent valid-use constraints rather than machine-state invariants. The full breakdown is $30 + 6 + 7 + 3 + 0 = 46$, all Qed, zero structural gaps. `RTLGapRegistry.v` records `rtl_gap_count = 0`.

The 46/46 proof coverage (all Qed) is the correct relationship between an abstract ISA and its physical instantiation. The abstract model defines what is correct; the hardware implements all of it; all 46 opcodes have formal bisimulation proofs. This is not a deficiency. It is honest proof accounting.

The abstract Thiele Machine is the Coq kernel. The hardware is a finite physical instantiation with explicit table sizes and word widths. The proofs state the bridge between them under the finite representation invariants where those invariants are needed. This is the only honest relationship physical hardware can have with an abstract mathematical model.

14 Algebraic Robustness: Binary Game Statistics and the μ-Ledger

The Thiele Machine has an opcode called CHSH_TRIAL that records outcomes of a 2-player binary game into hardware witness counters. I used it to stress-test the μ-ledger's cost algebra against a known algebraic boundary: the NPA moment matrix conditions for bounding what correlations are achievable under polynomial constraints.

I did not set out to find what I found. The μ-ledger's column-contractivity conditions — three polynomial inequalities that tell the machine when a CHSH correlation is honest — turn out to exactly characterize the zero-marginal slice of that quantum realizability boundary. Proved in Coq with zero physics axioms, no quantum mechanics, no empirical premises. The algebra forced it.

This section builds the CHSH game from scratch so that result makes sense.

14.1 The Bell test setup, from scratch

Imagine two players, Alice and Bob, in separate rooms. They cannot communicate once the game starts. A referee sends Alice one of two questions ($a \in \{0,1\}$). The referee sends Bob one of two questions ($b \in \{0,1\}$). Alice answers $x \in \{0,1\}$. Bob answers $y \in \{0,1\}$.

The game rule: they want $x \oplus y = a \wedge b$. The symbol $\oplus$ is XOR (exclusive OR): $x \oplus y = 1$ when $x \neq y$, and 0 when $x = y$. The symbol $\wedge$ is AND: $a \wedge b = 1$ only when both $a = 1$ and $b = 1$, otherwise 0. In plain language: their answers should be different if and only if both questions were 1. For all other question combinations (both 0, or one each), they should match.

Why 75% is the classical ceiling. Before the game, Alice and Bob can agree on any deterministic strategy: a function $x(a, \lambda)$ for Alice and $y(b, \lambda)$ for Bob, where λ is any shared randomness they agree on in advance. Once the game starts, no communication.

Fix any deterministic strategy and freeze λ. Then Alice has two predetermined answers, x_0 and x_1, and Bob has two predetermined answers, y_0 and y_1. Winning all four question combinations would require:

$$x_0 \oplus y_0 = 0, \quad x_0 \oplus y_1 = 0, \quad x_1 \oplus y_0 = 0, \quad x_1 \oplus y_1 = 1.$$

Now XOR the four left sides. Every variable appears twice, so the total is 0. XOR

the four right sides and the total is 1. That is impossible. So every deterministic local strategy loses at least one of the four cases, and the best deterministic strategy wins at most 3 out of 4 question combinations.

Randomized strategies cannot do better: they are just mixtures of deterministic strategies, and averaging strategies that each win 75% at most gives you 75% at most. This is not a vague claim. It is a direct consequence of linearity of expectation.

So: maximum classical win rate = 3/4 = 75%. Proven by exhaustion in `ClassicalBound.v` (zero Admitted): an executable trace using only PNEW, PSPLIT, and CHSH_TRIAL with $\mu = 0$ achieves exactly this, and `MinorConstraints.v` proves no $\mu = 0$ program can exceed it.

14.2 The CHSH inequality: where it comes from

Now here is what took me a while to understand: why is there a specific algebraic combination S that detects whether correlations can be explained classically?

The key move is this. For any fixed shared randomness λ, Alice's output $A_a(\lambda) \in \{-1, +1\}$ (I'm using ± 1 encoding now, not 0/1) and Bob's $B_b(\lambda) \in \{-1, +1\}$. Consider the expression:

$$A_0(\lambda)B_0(\lambda) + A_0(\lambda)B_1(\lambda) + A_1(\lambda)B_0(\lambda) - A_1(\lambda)B_1(\lambda)$$

Factor it:

$$= A_0(\lambda)(B_0(\lambda) + B_1(\lambda)) + A_1(\lambda)(B_0(\lambda) - B_1(\lambda))$$

Since B_0 and B_1 are each ± 1, either $B_0 + B_1 = \pm 2$ and $B_0 - B_1 = 0$, or $B_0 + B_1 = 0$ and $B_0 - B_1 = \pm 2$. The two terms cannot both be large. In either case, $|B_0 + B_1| + |B_0 - B_1| = 2$, and since $|A_0|, |A_1| \leq 1$, the entire expression is at most 2 in absolute value for any fixed λ.

Average over λ:

$$S = \langle A_0B_0 \rangle + \langle A_0B_1 \rangle + \langle A_1B_0 \rangle - \langle A_1B_1 \rangle$$

where $\langle A_aB_b \rangle$ is the correlation (expected value of the product) when questions are a and b. Since the expression before averaging was bounded by 2 for every λ, the average satisfies $|S| \leq 2$.

That is the argument. The combination with the minus sign is chosen exactly because the factoring trick works: one of the B-terms goes to zero and the other to ± 2, forcing

the whole thing below 2. The inequality $|S| \leq 2$ is not a coincidence. It is a pure algebraic consequence of the outputs being ± 1 and being determined locally.

This is proven in Coq in `MinorConstraints.v` (zero Admitted, `local_box_CHSH_bound`). The formal statement: for any factorizable box B, $|S(B)| \leq 2$. ClassicalBound.v proves the complementary result: that $|S| = 2$ IS achievable at $\mu = 0$ (a constructive witness showing the bound is tight).

14.3 The quantum bound: why $2\sqrt{2}$?

Quantum mechanics allows $|S|$ to reach $2\sqrt{2} \approx 2.828$. Why that number?

Starting from zero quantum mechanics:

In quantum mechanics, a particle can be in a superposition of states. For a qubit — the quantum version of a single bit — the state is a vector $\alpha|0\rangle + \beta|1\rangle$ where α and β are complex numbers satisfying $|\alpha|^2 + |\beta|^2 = 1$. You can picture this geometrically: any valid qubit state sits on the surface of a unit sphere (the Bloch sphere), with $|0\rangle$ at the north pole and $|1\rangle$ at the south pole.

A measurement on a qubit corresponds to choosing an axis through the sphere. The measurement returns $+1$ if the state is on one side of that axis, -1 if it's on the other. For a shared two-qubit state (Alice holds one qubit, Bob holds the other), the correlation between Alice's answer and Bob's answer depends on the angle between their measurement axes. The farther apart the axes, the less correlated the answers.

The $2\sqrt{2}$ bound comes from optimizing this geometry. Here is the derivation for the anticommuting optimal configuration.

For a qubit (a 2-state quantum system), any measurement observable with outcomes ± 1 satisfies $M^2 = I$ (the identity matrix) — squaring a ± 1-valued operator gives the identity. At the optimal configuration, Alice's two observables anticommute: $\hat{A}_0\hat{A}_1 = -\hat{A}_1\hat{A}_0$, and Bob's two observables also anticommute. This is the "90 degrees apart" condition on the Bloch sphere: $\hat{A}_0 = \sigma_x$, $\hat{A}_1 = \sigma_y$, for example. In this anticommuting configuration, all cross terms in $\hat{S}^2$ that involve anticommutators $\{\hat{A}_i, \hat{A}_j\} = \hat{A}_i\hat{A}_j + \hat{A}_j\hat{A}_i$ vanish (because anticommuting observables have anticommutator zero), and we get:

$$\hat{S}^2 = 4I \otimes I - [\hat{A}_0, \hat{A}_1] \otimes [\hat{B}_0, \hat{B}_1]$$

where $[\hat{A}_0, \hat{A}_1] = \hat{A}_0\hat{A}_1 - \hat{A}_1\hat{A}_0$ is the commutator. The commutator of two anticommuting ± 1 observables has operator norm exactly 2. So $\|[\hat{A}_0, \hat{A}_1]\| = 2$ and $\|[\hat{B}_0, \hat{B}_1]\| = 2$, giving $\|[\hat{A}_0, \hat{A}_1] \otimes [\hat{B}_0, \hat{B}_1]\| = 4$. Therefore $\|\hat{S}^2\| = 4 + 4 = 8$ and $\|\hat{S}\| = \sqrt{8} = 2\sqrt{2}$.

(For non-anticommuting configurations, the cross terms don't cancel perfectly and

you get a strictly smaller bound. The anticommuting configuration is the only one that saturates $2\sqrt{2}$.)

The Coq proof (`TsirelsonQuantumModel.v`) covers the zero-marginal NPA model specifically, not the full operator-algebra derivation above. Both give $|S| \leq 2\sqrt{2}$.

The NPA model here refers to a mathematical framework introduced by Navascués, Pironio, and Acín in 2007 for bounding quantum correlations. The idea: instead of directly characterizing what quantum mechanics allows, the NPA hierarchy gives a sequence of conditions (each tighter than the last) that any quantum correlation must satisfy. The "zero-marginal" version imposes the specific constraint that marginal probabilities factor correctly: the probability of Alice's outcome does not depend on Bob's measurement setting and vice versa, which is the no-signaling condition. A trace is `quantum_realizable` in this model if its CHSH statistics are consistent with those NPA-level constraints. This is a sufficient condition for quantum realizability, not a characterization of all quantum strategies.

`TsirelsonUpperBound.v` supplies the rational threshold bookkeeping and the algebraic/classical side of the story. I'm not claiming that `TsirelsonUpperBound.v` alone proves every possible quantum strategy is bounded. The formal claim is the named quantum-model theorem, and it has the `quantum_realizable` premise.

The gap between 2 and $2\sqrt{2}$ is real. No locally factorizable strategy can bridge it. That is the signature of the NPA realizability boundary: correlations that cannot be produced by any shared classical randomness, no matter how much pre-game coordination you allow. Under the named hypothesis A_QM (that quantum experiments realize the NPA model), this gap corresponds to the regime accessible only to quantum strategies. The algebraic fact holds regardless.

The Inquisitor's note on the rational approximation. `TsirelsonUpperBound.v` names the rational threshold $5657/2000 \approx 2.8285$ for exact rational comparisons. The real-valued quantum-model bridge uses $\sqrt{8}$ directly. Those are different proof surfaces, and the manuscript should not blur them.

14.4 What the cost algebra proves

Theorem 14.1 (Classical CHSH Bound)**.** *For any locally factorizable correlation (Alice's answer depends only on a and shared randomness λ; Bob's answer depends only on b and λ), $|S| \leq 2$. And $|S| = 2$ is achievable at $\mu = 0$, so the bound is tight.*

Theorem 14.2 (Non-locality of CHSH violation)**.** *WitnessCount configurations with $S > 2$ cannot be produced by any local hidden variable model.*

Theorem 14.3 (Tsirelson upper bound for the zero-marginal NPA model)**.** *If a trace is quantum-realizable in the zero-marginal NPA model, then its CHSH value satisfies $|S| \leq \sqrt{8} = 2\sqrt{2}$.*

Theorem 14.4 (General algebraic Tsirelson bound)**.** *For every algebraically coherent correlator — one satisfying $|E_{xy}| \leq 1$ and four constraints derived from 3×3 submatrices of*

the NPA moment matrix ("minors" are the determinants of square submatrices; requiring them to be non-negative is part of the PSD condition), with witness parameters t, s — the CHSH value satisfies $S^2 \leq 8$, i.e., $|S| \leq 2\sqrt{2}$. This is the general *case: all algebraically coherent correlators, not just the symmetric configuration. The Coq proof uses* `psatz`*, a tactic that finds a certificate showing a polynomial inequality holds by constructing an explicit sum-of-squares decomposition — a way of writing the inequality as a sum of squared polynomial expressions, which is manifestly non-negative. The rational approximation corollary uses* $5657/2000$ *as an exact rational overestimate of* $2\sqrt{2}$ *(verifiable:* $(5657/2000)^2 = 32001649/4000000 > 8$*), allowing the bound to be checked in exact rational arithmetic without floating point. No physics premises. Pure polynomial arithmetic. The Tsirelson bound* $2\sqrt{2}$ *is derived from algebra, not assumed.*

The coupling language version of Bell's theorem is proved in `CHSHCouplingBridge.v`: a WitnessCount-consistent locally deterministic strategy (a0, a1, b0, b1) produces a *separable coupling* — one that maps each setting to exactly one outcome type. Any locally factorizable morphism coupling satisfying WCLocallyConsistent has $|S| \leq 2$. Contrapositively: $S > 2$ rules out every locally factorizable morphism coupling. The no-go is stated in the morphism coupling language, not just in the statistical layer.

What does this have to do with computation? The connection is through the witness counters and the certification layer. If a Thiele Machine program accumulates CHSH trial results (via CHSH_TRIAL) and then certifies that $S > 2$ (via CERTIFY or LASSERT), the certification is non-free. Certifying binary game statistics requires paying $\mu \geq 1$. The statistics themselves (raw CHSH trials) are free. Claiming you know what they mean is not. This is the witness insight taxonomy (Section 5).

14.5 The bit-commitment requirement

Obtaining attested cross-correlations above the classical bound requires a disclosure instruction. Here is why that is a structural fact, not a policy choice.

Each CHSH_TRIAL records setting bits and outcome bits into witness counters. Those raw counts are free. But certifying what they mean — attesting that the cross-correlations $\langle A_a B_b \rangle$ justify a particular claim — requires explicit certification, and certification requires an explicit cost.

Proved in `RevelationRequirement.v`. The structural fact: the only instruction that can set `csr_cert_addr` nonzero is MORPH_ASSERT (when the morphism exists and the property string is non-empty). Any trace that ends with supra_cert must include a valid MORPH_ASSERT step. The raw data (CHSH_TRIAL counts) is free. The certified claim about what the data means is not.

14.6 Algebraic characterization of the μ-ledger honesty condition

Here is the stress-test result. I did not design the μ-ledger's column-contractivity conditions to match anything from physics. I defined them as the natural polynomial

conditions that make the NPA moment matrix well-formed. The algebra forced the following.

Positive semidefinite (PSD): A matrix is positive semidefinite if for any vector v you can feed it, the result $v^T Mv \geq 0$. That is, the matrix never maps a vector to something pointing "backwards" relative to the vector. For a 2×2 matrix $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$, this means $a \geq 0$, $c \geq 0$, and $ac - b^2 \geq 0$ (the diagonal entries are non-negative and the determinant is non-negative). For larger matrices, it generalizes: every principal submatrix (every square subblock on the diagonal) must also be PSD. PSD is the mathematical condition that a matrix can represent genuine inner products, covariances, or correlations in a quantum state. The NPA framework uses PSD of a specific 5×5 matrix (the "moment matrix") as the condition for quantum realizability. You build that 5×5 matrix from the four CHSH correlators, and if it's PSD, the correlators are consistent with some quantum state.

14.6.1 The matrix, written out

Here is the actual 5×5 NPA moment matrix for the zero-marginal CHSH scenario. The five operators are: I (identity), A_0 (Alice's measurement for setting 0), A_1 (Alice's measurement for setting 1), B_0 (Bob's for setting 0), B_1 (Bob's for setting 1). The (i, j) entry is the expected value of $O_i \cdot O_j$. Zero-marginal means we set $\langle A_0 \rangle = \langle A_1 \rangle = \langle B_0 \rangle = \langle B_1 \rangle = 0$ and the cross-correlation terms $\langle A_0 A_1 \rangle = \langle B_0 B_1 \rangle = 0$.

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix}$$

Rows and columns are indexed 0–4 for $\{I, A_0, A_1, B_0, B_1\}$. The diagonal is all 1s because $O_i^2 = I$ for ±1-valued observables. The off-diagonal entries E_{ab} are the CHSH correlators. Everything else is zero because the marginals vanish.

14.6.2 Where the three conditions come from

The three column-contractivity conditions come directly from PSD of this matrix. Here is the derivation.

Condition 1: $1 - E_{00}^2 - E_{10}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1, 2, 3\}$ (the A_0, A_1, B_0 block):

$$\begin{pmatrix} 1 & 0 & E_{00} \\ 0 & 1 & E_{10} \\ E_{00} & E_{10} & 1 \end{pmatrix}$$

Compute the determinant by expanding along the first row:

$$1 \cdot (1 \cdot 1 - E_{10}^2) - 0 + E_{00} \cdot (0 - E_{00}) = 1 - E_{10}^2 - E_{00}^2$$

For PSD this determinant must be ≥ 0. That is condition 1.

Condition 2: $1 - E_{01}^2 - E_{11}^2 \geq 0$

Take the 3×3 submatrix from rows and columns $\{1, 2, 4\}$ (the A_0, A_1, B_1 block):

$$\begin{pmatrix} 1 & 0 & E_{01} \\ 0 & 1 & E_{11} \\ E_{01} & E_{11} & 1 \end{pmatrix}$$

Same calculation: determinant $= 1 - E_{01}^2 - E_{11}^2 \geq 0$. That is condition 2.

Condition 3: $(1 - E_{00}^2 - E_{10}^2)(1 - E_{01}^2 - E_{11}^2) \geq (E_{00}E_{01} + E_{10}E_{11})^2$

This comes from the 4×4 block of rows/columns $\{1, 2, 3, 4\}$. A matrix is PSD if and only if you can write it as a product $C^T C$ for some matrix C (the Cholesky decomposition), or equivalently, its Schur complement is PSD. The Schur complement of the top-left 2×2 identity block is:

$$S = I_2 - C^T C, \quad \text{where } C = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix}$$

$$S = \begin{pmatrix} A & -B \\ -B & C \end{pmatrix}$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. For S to be PSD: diagonal entries non-negative (conditions 1 and 2 again) and determinant $AC - B^2 \geq 0$. That is condition 3.

That is the complete derivation. Three conditions. Two from 3×3 submatrices. One from the Schur complement of the 4×4 block. All of it falls out of the single requirement that M is positive semidefinite.

Why this equals quantum realizability. The forward direction is the derivation above: if the correlators are column-contractive, the matrix M is PSD by construction. The reverse direction (proved in `ThieleQuantumEquivalence.v`) is that if M is PSD, the correlators must be column-contractive. The proof: take any PSD matrix M and feed it the specific vector $v = (0, -(E_{00}y_0 + E_{01}y_1), -(E_{10}y_0 + E_{11}y_1), y_0, y_1)$. The PSD condition $v^T M v \geq 0$ gives the quadratic form $Ay_0^2 - 2By_0y_1 + Cy_1^2 \geq 0$ for all real y_0, y_1. A quadratic $ay^2 + 2by + c \geq 0$ for all y if and only if $ac \geq b^2$ (non-negative discriminant). That closes condition 3. The diagonal conditions come from setting $y_1 = 0$ and $y_0 = 0$ separately.

A correlation $(E_{00}, E_{01}, E_{10}, E_{11})$ is *Thiele-honest* if and only if it is zero-marginal NPA realizable. Exactly. Not approximately. Not "consistent with some physics axiom." The biconditional is pure polynomial arithmetic, zero axioms:
Theorem 14.5 (Thiele honesty $\iff$ NPA-PSD)**.**

$$\textit{honest}(\mathbf{E}) \iff \textit{psd-npa}(\mathbf{E})$$

where honest *means the three column-contractivity conditions hold, and* psd-npa *means the* 5×5 *zero-marginal NPA matrix is positive semidefinite.*

The left side is three polynomial inequalities (column contractivity of the zero-marginal NPA matrix):

$$1 - E_{00}^2 - E_{10}^2 \geq 0 \quad \text{(condition 1)}$$

$$1 - E_{01}^2 - E_{11}^2 \geq 0 \quad \text{(condition 2)}$$

$$AC \geq B^2 \quad \text{(condition 3)}$$

where $A = 1 - E_{00}^2 - E_{10}^2$, $B = E_{00}E_{01} + E_{10}E_{11}$, $C = 1 - E_{01}^2 - E_{11}^2$. These are the three conditions of `column_contractive`: column 0 norm ≤ 1, column 1 norm ≤ 1, determinant $AC - B^2 \geq 0$. The right side says the NPA moment matrix is positive semidefinite — the standard mathematical characterization of quantum realizability.

The forward direction (`A_QM_thiele`) was proved first. The reverse direction is new: given a PSD NPA matrix, specialize it at the vector $(0, -(E_{00}y_0 + E_{01}y_1), -(E_{10}y_0 + E_{11}y_1), y_0, y_1)$. The PSD condition gives a quadratic-in-y_0/y_1 that is everywhere non-negative. Then `quadratic_nonneg_discriminant` ($\forall t, a + 2bt + ct^2 \geq 0 \Rightarrow b^2 \leq ac$) closes the determinant condition. The diagonal conditions come from $y_0 = 1, y_1 = 0$ and $y_0 = 0, y_1 = 1$.

What this means for the hierarchy of correlations:

- Classical local hidden variables: $|S| \leq 2$.
- Thiele-honest = zero-marginal NPA realizable (biconditional, both directions, zero axioms). Every Thiele-honest correlation satisfies $|S| \leq 2\sqrt{2}$, but NOT every correlation with $|S| \leq 2\sqrt{2}$ is Thiele-honest: $(1, 0, 1, 0)$ has $|S| = 2$ but fails condition 1.
- No-signaling: no constraint beyond $|E_{ab}| \leq 1$.
- PR box ($E_{00} = E_{01} = E_{10} = 1$, $E_{11} = -1$): condition 1 gives $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Fails immediately. Proved in `pr_box_not_thiele_honest`.
- Tsirelson bound: `tsirelson_from_thiele_honesty` proves Thiele-honest $\Rightarrow |S| \leq 2\sqrt{2}$. The contrapositive ($|S| > 2\sqrt{2} \Rightarrow$ not Thiele-honest) is also proved. The converse is false and not proved.

A Turing machine can output the correlations $(1, 0, 1, 0)$. These

are not Thiele-honest: condition 1 gives $1 - 1^2 - 1^2 = -1 < 0$. The proof in Coq (`turing_machine_has_free_correlations`, `ThieleQuantumEquivalence.v`) destructs on condition 1 and closes by `lra`. The μ-ledger's column-contractivity conditions are what make the quantum boundary real for computation: there exist correlations a Turing machine certifies but no Thiele Machine certifies as honest.

To falsify: exhibit $E_{00}, E_{01}, E_{10}, E_{11}$ satisfying the three polynomial inequalities but with a non-PSD NPA matrix. No such example exists; the theorem proves it impossible. Or exhibit a quantum experiment producing a non-PSD NPA matrix. That would break quantum mechanics, not this theorem.

15 Irreversibility Accounting and the μ-Hierarchy

15.1 Historical motivation: Landauer's irreversibility argument

Rolf Landauer proved in 1961 that erasing one bit of information from a physical system must release at least $kT \ln 2$ joules of heat into the environment. The μ-ledger's irreversibility indicator was inspired by this argument. The formal Coq content requires none of it.

A bit of information is stored in a physical system with two distinguishable states: call them 0 and 1. "Erasing" the bit means forcing the system to state 0 regardless of what it was before. Before erasure, the bit could be 0 or 1; you have no idea which. After erasure, you know it is 0. You have gone from maximum uncertainty (1 bit of entropy) to zero uncertainty.

Here is the key physical constraint: entropy — loosely, disorder, or uncertainty — can only increase in a closed system. That is the second law of thermodynamics. The universe keeps a running total of disorder, and that total never goes down. Erasing the bit reduced the bit's uncertainty by $k \ln 2$ (here k is Boltzmann's constant, a physical conversion factor between temperature and energy, approximately 1.38×10^{-23} J/K). Some other part of the universe must absorb at least that much new uncertainty to compensate. The cheapest place to dump entropy is as heat into a thermal environment at temperature T. The heat required to compensate is $Q = kT \ln 2$.

I found this argument important because it is the same kind of argument as No Free Insight: you cannot get structure for free, and the machine tracks where you paid. The physical connection (that μ counts correspond to actual thermodynamic entropy) is a named bridge hypothesis (`mu_landauer_unruh_calibrated`): it calibrates μ to physical energy units. That hypothesis is not proven in Coq. The formal theorem below requires none of it.

15.2 What the Thiele Machine proves

Theorem 15.1 (μ-Landauer Step Bound)**.** *For any instruction i, the `irreversible_bits i` indicator (a lower-bound estimate of how many bits of ir-*

reversible erasure the instruction performs) is bounded by `instruction_cost i`*. "Conservative" here means the indicator never overcounts the actual irreversibility — it is a floor, not a ceiling. For a single step, the μ increase is at least* `irreversible_bits i`*; for a trace, total irreversible-bit indicators are bounded by total instruction cost.*

This is a theorem about the formal step semantics of the machine. It says the machine's cost accounting is Landauer-shaped: every positive-cost irreversible indicator used by this file is covered by μ. It is deliberately coarse. It is not a microscopic physical erasure model for every possible hardware transition.

The physical interpretation, that μ costs correspond to actual thermodynamic entropy, is conditional. It depends on a named hypothesis (`mu_landauer_unruh_calibrated`) that calibrates μ to physical energy units. That hypothesis is physically motivated and consistent with the formal development, but it is not proven in Coq. It is a bridge to physics, not a derived fact. I say this directly because it is true.

The formal content of `LandauerDerivation.v` is narrower and cleaner: (1) only CERTIFY sets `vm_certified := true`, proven by case analysis on every instruction; (2) CERTIFY's cost is $S(\delta) \geq 1$ by construction; (3) therefore certification always requires positive μ-cost; (4) total μ-cost bounds the file's `total_irreversible_bits` measure over the trace. `irreversible_bits` is a conservative indicator function: it returns 1 for instructions that the file classifies as irreversible (those in the certification/revelation class) and 0 for everything else. The theorem says: the total number of irreversible-class instruction firings in a trace is bounded by the total μ spent. This is the formal Landauer-shaped claim: irreversible operations are tracked by μ. It does not say that irreversible operations in the thermodynamic sense (erasing physical bits) cost exactly $kT \ln 2$ each. That would require the calibration hypothesis. None of this requires a physical calibration. Physical calibration would plug in the Boltzmann constant and the temperature, turning μ counts into joules. That step is the named bridge hypothesis.

15.3 The μ-hierarchy

The proof:

The μ-cost hierarchy theorem is the time hierarchy theorem analogue for this machine. It says μ-budget k is necessary and sufficient for level-k certification. The separation is strict and infinite.

Theorem 15.2 (μ-Hierarchy)**.** *For every $k \geq 1$:*

1. *There exists a trace reaching* `vm_certified = true` *with μ-cost exactly k.*
2. *Any trace with* `vm_mu` $\geq k$ *has* `trace_mu_cost` $\geq k$*.*

Corollary 15.3 (`mu_hierarchy_no_upper_bound`)**.** *No fixed μ-budget suffices for all certification levels.*

The point: there is no shortcut. You cannot certify at level k for less than k units of μ. Paying more gets you more. Paying less is structurally impossible. This is a hierarchy theorem: each μ-cost level is strictly more powerful than the last, and no level is achievable for less than its stated price.

The analogy is the time hierarchy theorem from classical complexity theory. That theorem says: for any time bound $T(n)$, there are problems solvable in time $T(n)^2$ that are not solvable in time $T(n)$. More time buys strictly more computation, and no amount of cleverness closes the gap. The μ-hierarchy theorem says the same thing for structural certification cost: more μ buys strictly more certified structure, and no amount of cleverness reaches level k for less than k units of μ.

To falsify: find $k \geq 1$ and a trace reaching `vm_certified = true` and `vm_mu` $\geq k$ with total trace cost $< k$. The theorem says this cannot happen. If it does, the cost accounting is wrong.

16 Discrete Geometry over Partition Graphs

The partition graph is also a discrete geometric object. Here is one result that falls out, no physics required.

16.1 Discrete triangulation of the partition graph

Each module in the partition graph carries structural data, and several bridge files use μ-derived quantities as geometry-like weights. The discrete Gauss-Bonnet file is more specific than that: it reads the partition graph as a triangulated combinatorial object and uses an equilateral angle model where every triangle corner contributes $\pi/3$. It does not derive those angles directly from μ costs.

A discrete triangulation defines a curvature-like angle defect. To see why: on a flat surface, if you draw a triangle, the interior angles sum to $180°$. On a curved surface, the angles can sum to more or less than $180°$. Curvature is the amount by which triangles deviate from flat. In the equilateral discrete model, curvature at a vertex is the defect left after subtracting the incident triangle angles from 2π.

The Euler characteristic χ is a number that captures the overall shape of a triangulated surface, independent of how you draw the triangles. For any triangulation with V vertices, E edges, and F triangular faces: $\chi = V - E + F$. This is remarkable: no matter how you triangulate the same surface, you get the same number. For a sphere, $\chi = 2$. For a torus (donut shape), $\chi = 0$. The continuum Gauss-Bonnet theorem says the total curvature of a closed surface equals $2\pi\chi$: the local geometric information (curvature) integrates to a global topological fact (shape). That is the theorem I proved in the discrete setting.

16.2 The discrete Gauss-Bonnet theorem (unconditional)

Theorem 16.1 (Discrete Gauss-Bonnet)**.** *For any* `well_formed_triangulated` *partition graph g in the equilateral angle model,*

$$\text{angle\_defect}(g) = 5\pi \cdot \chi(g)$$

This is a fully proven theorem about the discrete triangulation model used here. It is not the continuum Gauss-Bonnet theorem and it is not a derivation of angles from μ cost. It is a combinatorics result: any well-formed triangulated partition graph satisfies the angle-defect formula. That is the full extent of the geometry section.

17 What Coq is, and why I used it

Coq is a proof assistant. It is a programming language where the type system is so expressive that you can write proofs as programs and have the compiler check whether they are valid.

Here is the idea. In a normal programming language, a type says something about what kind of value a variable holds. In Python, `x: int` means x is an integer. In Coq, types are propositions. The type `n > 0` means "the proof that n is positive." A function from `n > 0` to `n + 1 > 0` is a proof that if n is positive, $n + 1$ is too.

The compiler checks whether the types line up. If your proof has a hole and you write "trust me," the compiler rejects it. There is no way to convince the compiler of a false theorem by being persuasive or authoritative. The only way through is to actually prove it.

I used Coq because I don't trust informal arguments. Including my own. I spent months being convinced by my own reasoning that various claims were true, only to find when I tried to write the Coq proof that some key step was wrong. The machine caught it. That is the point.

17.1 The Inquisitor

Beyond Coq's type checker, the project has a custom proof auditor called the Inquisitor (`scripts/inquisitor.py`). It runs on every commit and has 160 distinct rules. The main categories:

- **No holes.** No `Admitted`, `admit`, or `give_up` anywhere in the active proof files. No unproved `Axiom` or `Parameter` declarations in project code.
- **No vacuous theorems.** No theorem whose conclusion is literally `True`, `0 = 0`, or any other tautology that proves nothing. No "theorem" of the form $P \to P$. No existence claims backed by constant witnesses. Coq's kernel already prevents circularly dependent proofs (it checks definitional well-foundedness); the Inquisitor catches separately the case where a proof is technically valid but

epistemically empty — proving `True` by `trivial`, for instance.

- **No physics stubs.** No physics quantity defined as a constant placeholder. No physics claim without a corresponding invariance condition. No theorem stated about a physics concept if that concept is not genuinely formalized.
- **No trivial true proofs.** If a theorem's entire proof is `trivial` or `tauto` and the conclusion reduces to `True`, the theorem is rejected.
- **Scope enforcement.** Kernel files (Tier 1) may not import exploratory files. Theorem connectivity: every file must reach the cost or semantics foundation through its import chain.
- **Comment hygiene.** No TODO/FIXME/WIP markers in active proof comments.

Current Inquisitor status: zero HIGH/MEDIUM/LOW findings across all 208 Coq files in the active tree.

18 Zero Admitted: what it actually means

"Zero Admitted" means: every theorem in the proof tree was derived without any gaps. No step was asserted without a derivation. The machine verified the complete chain.

This is what it does NOT mean:

- It does not mean the theorems prove physical reality. The theorems prove things about the Thiele Machine model, which is a formal object in Coq. Whether the model correctly describes physical reality is a separate empirical question.
- It does not mean the Coq axioms themselves are consistent. Coq is built on a formal foundation called the Calculus of Inductive Constructions. The key idea underlying it is the Curry-Howard correspondence: proofs and programs are the same thing. A proof that P is true is literally a program of type P. Checking the proof is the same as type-checking the program. The "inductive" part means you can define your own recursive types — natural numbers, lists, trees — inside the system instead of having them as magical primitives. This matters because it makes the foundation explicit and auditable.
  Now here is the honest limitation. We believe this foundation is logically consistent — that you cannot derive a contradiction from it — but this has not been proven within Coq itself. That is not a weakness in Coq specifically; it is a fundamental theorem by Kurt Gödel from 1931. Gödel proved that any formal system powerful enough to do arithmetic cannot prove its own consistency from within itself. If it could, it would be contradictory. So Coq's foundation is trusted, not proved-from-within, which is the honest state of affairs for any powerful formal system. The Coq community has studied the foundation extensively and found no inconsistencies. But "zero Admitted" means every theorem follows

from those axioms. It does not mean those axioms are themselves in-system guaranteed.

- It does not mean the hardware implements exactly the Coq semantics in all cases. All 46 opcodes have Qed bisimulation proofs (30 fully unconditional, 6 with argument-validity conditions, 7 unconditional for KamiReachable states, 3 with semantic-level conditions, zero structural gaps); documented in `RTLGapRegistry.v` and `CloseoutVerification.v`.
- It does not mean the OCaml runner binary is proven inside Coq to be bitwise identical to the Coq semantics. `OCamlExtractionBridge.v` proves Coq-side observable facts and the parity suite checks the extracted binary empirically.

The kernel tier (Section 7) is the strongest tier. The theorems listed there have zero Admitted and zero named hypotheses. They are machine-checked derivations from the Coq type theory axioms and the Thiele Machine definitions. Nothing else.

The bridge tier involves named hypotheses and documented trust boundaries: the BSC compiler trust boundary for hardware, the A_QM physical assumption for the quantum connection, and the external OCaml parity boundary. These are explicitly labeled and auditable. They are not Admitted proofs. They are documented limits of the formal claim.

19 How to break this

Every claim in this document has a falsification condition. Here are the main ones.

Falsify No Free Insight.
Find a Thiele Machine program trace that:

- starts with `vm_certified = false` and `vm_mu = 0`
- ends with `vm_certified = true`
- ends with `vm_mu = 0`

If you find such a trace, the Coq proof of `certification_requires_positive_mu` is wrong and the compilation will fail when you add the counterexample as a theorem.

Falsify μ-monotonicity.
Find a state s and instruction i such that $(\texttt{vm\_apply}\ s\ i).\mu < s.\mu$. The Coq proof of `vm_apply_mu` will fail.

Falsify categorical separation.
Prove that for ALL states s_1, s_2 with identical classical shadow, ALL instructions produce the same error behavior. This would contradict `categorical_separation` in Coq, meaning the proof has an error.

Falsify Turing strictness.
Prove that every Thiele program can be simulated by a classical program with the same observable behavior (registers, memory, μ) without ever touching the partition graph or morphism map. This would contradict `D5_thiele_strictly_extends_classical`.

Falsify structural entitlement.
Find two Thiele states with the same classical shadow where no probe can distinguish their structural fields. That would contradict `shadow_strictly_lossy`. Or produce a strict feasible-set subset with a distinguishing observation that does not yield stronger membership predicates; that would contradict the feasible-subset theorem.

Falsify universal certification cost.
Build a `CertificationSystem` satisfying the one-step rule `cs_cert_costs`, then give a trace from uncertified to certified with total cost 0. That would contradict `universal_nfi_any_substrate`. If you instead drop the one-step rule, you have found a free-certification system, not a counterexample.

Falsify the classical CHSH bound.
Exhibit a locally factorizable strategy that achieves $|S| > 2$. This would contradict `chsh_stat_violation_not_local` and also violate known mathematics.

The Inquisitor standard.
Run `python scripts/inquisitor.py` on the repository. If it finds any HIGH/MEDIUM/LOW finding, the proof hygiene has degraded. Current status: zero findings. If that changes, it is a bug.

20 The full claim ledger

These are the kernel-tier claims (machine-checked, no named hypotheses):

1. *no_free_certification*: csr_cert_addr going $0 \to$ nonzero in one step requires cost ≥ 1.
2. *no_free_certification_mu*: same, with μ increment ≥ 1.
3. *no_free_certification_trace_mu*: trace-level version.
4. *no_free_certification_certified*: vm_certified going false$\to$true requires cost ≥ 1.
5. *certification_requires_positive_mu*: both channels covered.
6. *universal_nfi_any_substrate*: any abstract certification system satisfying the one-step cost rule has trace-level non-free certification.
7. *thiele_universal_nfi_cert_addr*: universal theorem instantiated for Thiele's `csr_cert_addr` channel.
8. *thiele_universal_nfi_certified*: universal theorem instantiated for Thiele's `vm_certified` channel.

9. *thiele_represents_simulating_cert_system*: any certification system with a simulation morphism into Thiele transfers certification into a Thiele certified execution.
10. *forget_kernel_is_eq_on_classical*: the classical snapshot quotient forgets exactly the non-classical fields.
11. *blindness_non_injective*: distinct Thiele states can have the same classical snapshot.
12. *certification_is_lost*: certification can be in the kernel of the classical forgetful map.
13. *shadow_strictly_lossy*: the classical shadow forgets morphism structure.
14. *mu_ledger_necessity*: no strict-classical oracle on $(\texttt{mem}, \texttt{regs}, \texttt{pc})$ recovers the pair $(\mu, \texttt{certified})$.
15. *vm_mu_not_classically_determined* / *vm_certified_not_classically_determined*: neither ledger balance nor certification status is a function of strict classical state.
16. *mu_ledger_mutual_independence*: μ and `vm_certified` are each independent of strict classical state and of each other under cost-only/cert-only projections.
17. *mu_ledger_minimality* / *P_full_is_minimal_complete_extension*: the full $(\texttt{mem}, \texttt{regs}, \texttt{pc}, \mu, \texttt{certified})$ projection is complete, and dropping either μ or certification destroys the corresponding completeness.
18. *thiele_nfi_pc_indexed*: PC-indexed execution version.
19. *mu_is_initial_monotone*: μ is the unique canonical cost measure.
20. *vm_apply_mu*: exact ledger identity $(\texttt{vm\_apply}\ s\ i).\mu = s.\mu + \texttt{instruction\_cost}(i)$. The cost is not just δ — it is δ for ordinary instructions, $S(\delta)$ for cert-setters, $|\text{payload}|_{\text{bits}} + S(\delta)$ for EMIT, etc.
21. *kernel_certified_implies_positive_mu*: from zero, reaching certified implies $\mu > 0$.
22. *feasible-subset-implies-strict-predicates*: feasible-set narrowing plus a distinguishing observation yields a strictly stronger predicate.
23. *strengthening_requires_structure_addition*: certified predicate strengthening requires a structure-addition event.
24. *b4_information_reduction_derives_strict_predicates*: feasible-set reduction can generate the strict-predicate premise used by NoFI.
25. *info_priced_cert_executions_bound*: executed cert-setting events are bounded by $\Delta\mu$.
26. *obs-partition-to-posterior-witness*: observation classes package into the posterior-representative witness for structural entitlement.
27. *info-priced-posterior-reduction-bound*: with a decision-tree/representative witness, feasible-set compression is bounded by $\Delta\mu$.
28. *info-priced-weighted-feasible-bound*: the $\Delta\mu$ lower bound extends to nat-weighted feasible sets.
29. *structural_entitlement_representation*: strict narrowing plus distinguishability, certified posterior admissibility, and an explicit posterior-representative reduction imply a strictly stronger predicate, a structure-addition event, and a $\Delta\mu$ lower bound.
30. *every_sound_structural_shortcut_lands_here*: every packaged `SoundStructuralShortcut` instantiates the structural-entitlement representation theorem.

31. *categorical_separation*: classical-identical states distinguishable by morphism probe.
32. *classical_observer_cannot_separate*: no classical function separates morphism-distinct states.
33. *shadow_proj / shadow_separation_theorem*: formal surjection, strictly lossy.
34. *D2_faithfulness*: classical programs embedded faithfully in Thiele.
35. *D3_conservativity*: classical programs leave structural layer untouched.
36. *D4_strictness*: there exist Thiele steps that escape the classical fragment.
37. *D5_thiele_strictly_extends_classical*: Thiele strictly exceeds classical semantics.
38. *degenerate_projection_theorem*: shadow projection is the classical equivalence quotient.
39. *no_classical_certification_decider*: no classical machine can decide morphism-certification.
40. *chsh_stat_violation_not_local*: CHSH-violating statistics are non-local.
41. *structural_trace_preserves_cert_addr*: traces with no cert-address setter preserve `csr_cert_addr`.
42. *partition_refinement_nonfree*: certified partition knowledge cannot be free.
43. *partition_free_but_certification_nonfree*: partition structure can be built at $\delta = 0$, but certified partition knowledge cannot be acquired for free.
44. *witness_insight_nonfree_general*: certified non-local witness insight requires $\mu \geq 1$.
45. *witness_insight_complete_taxonomy*: full three-tier taxonomy proven.
46. *separable_coupling / non_separable_coupling*: Functional (separable) and non-functional (non-separable) coupling predicates with concrete witnesses (`CategoryLaws.v`).
47. *CHSH coupling bridge* (`CHSHCouplingBridge.v`): Bell's theorem in morphism coupling language. A locally consistent strategy produces a separable coupling and has $|S| \leq 2$. $S > 2$ rules out every locally factorizable morphism coupling.
48. *graph_certify_morphism_lookup*: morphism certification cost utility, including lookup correctness.
49. *exists_covering_tree*: for any feasible-set reduction, a covering complete binary tree exists constructively.
50. *info_priced_reduction_no_tree_hypothesis*: entropy bound without requiring the tree as an explicit input.
51. *sound_shortcut_from_components*: assembly lemma for SoundStructuralShortcut from its component witnesses.
52. *in_muP / in_muNP / classical_in_muP*: μ-complexity class definitions and basic inclusion.
53. *thiele_honest_iff_quantum_realizable* (`ThieleQuantumEquivalence.v`): Thiele honesty $\iff$ zero-marginal NPA realizability. A zero-axiom biconditional. Forward: column-contractive $\to$ NPA PSD. Reverse: NPA PSD $\to$ column-contractive (new; proved by vector specialization + quadratic discriminant). Zero-marginal NPA realizability is the orthogonal-observables slice of the quantum elliptope ($\rho_{AA} = \rho_{BB} = 0$), not the full elliptope. Classical correlations like $(1, 0, 1, 0)$ have $|S| \leq 2$ but fail column contractivity — the classical polytope and the Thiele-honest set intersect but neither contains the other.
54. *tsirelson_is_thiele_boundary*: Thiele-honest $\Rightarrow |S| \leq 2\sqrt{2}$ (forward direction), and

$|S| > 2\sqrt{2} \Rightarrow$ not Thiele-honest (the contrapositive). These are the same implication, stated both ways. The converse ($|S| \leq 2\sqrt{2} \Rightarrow$ Thiele-honest) is false and is not claimed.

55. *pr_box_not_thiele_honest*: PR box ($E_{00} = E_{01} = E_{10} = 1$, $E_{11} = -1$) fails the first column-contractivity condition: $1 - E_{00}^2 - E_{10}^2 = 1 - 1 - 1 = -1 < 0$. Direct arithmetic.
56. *super_quantum_not_thiele_honest*: $|S| > 2\sqrt{2}$ implies not Thiele-honest. Proved from the Tsirelson boundary.
57. *turing_machine_has_free_correlations*: Turing machines produce correlations Thiele Machines reject as dishonest. The μ-cost structure is what makes the quantum boundary computationally real.
58. *A_QM_thiele*: Thiele honesty implies quantum realizability. Proves A_{QM} from first principles, not assumed.
59. *tsirelson_from_thiele_honesty*: $|S| \leq 2\sqrt{2}$ follows from column contractivity. Zero axioms.
60. *mu_hierarchy_theorem* (`MuHierarchyTheorem.v`): for every $k \geq 1$, there exists a certified trace with cost exactly k, and every certified trace with `vm_mu` $\geq k$ has cost $\geq k$. Infinite strict μ-complexity separation.
61. *mu_hierarchy_no_upper_bound*: no fixed μ-budget suffices for all levels.
62. *KamiReachable invariants* (`HardwareReachability.v`): four hardware coupling/morph invariants proved inductive from hardware reset: `KamiReachable_morph_table_wf`, `KamiReachable_coupling_wf`, `KamiReachable_coupling_zero_empty`, `KamiReachable_coupling_desc_safe`.
63. *KamiReachable unconditional group* (`HardwareReachability.v`): MORPH_DELETE, MORPH_ASSERT, MORPH_GET, COMPOSE, MORPH_TENSOR, MORPH, MORPH_ID unconditional for any KamiReachable state. Key supporting lemmas: `KamiReachable_sizes_nonzero_bounded`, `snap_pt_sizes_zero_graph_none`.
64. *end_to_end_rtl_kernel_correctness* (`VerilogSemantics.v`): physical RTL implements Kami step semantics for KamiReachable states, given the named trust boundary `physical_rtl_matches_kami_step` (backed by 31/31 cosim + 11,049 fuzz tests).

Bridge-tier claims (machine-checked, with named hypotheses or documented trust boundaries):

- **Hardware bisimulation**: 46/46 opcodes with Qed proofs (30 fully unconditional + 6 argument-validity conditional + 7 unconditional for KamiReachable states + 3 semantic-boundary conditional, 0 structural gaps); documented in `RTLGapRegistry.v` (`rtl_gap_count = 0`) and `CloseoutVerification.v`.
- **OCaml extraction**: `ocaml_bisimulation_closure` proves NoFI, μ-monotonicity, and totality for the Coq-side extracted observable. The external OCaml binary equivalence is a parity-test trust boundary, not a kernel theorem.
- **Structural advantage**: `StructuralAdvantage.v` proves exact μ costs and the arithmetic time-tax facts for the factored-search witness; executable tests check

the measured VM behavior.

- **Cost formula forcing**: `cost_necessity` + `cost_uniqueness`, conditional on Landauer-Unruh calibration for physical interpretation.

21 What I don't know

Here is what I don't know.

Physical interpretation of μ. Whether the μ ledger actually corresponds to thermodynamic entropy in a physical Thiele Machine is an open empirical question. The formal development is consistent with Landauer's principle. I think it does. I cannot prove it.

Full informal representation theorem for structural entitlement. I've proved `structural_entitlement_representation` and `every_sound_structural_shortcut_lands_here`. For the explicit witness class, structural entitlement becomes strict predicate strengthening, requires structure addition, and carries a $\Delta\mu$ lower bound. I have not proven the open translation theorem: whether every human informal story about symmetry, factorization, modularity, independence, or decomposition can always be packaged as a `SoundStructuralShortcut`. That is the next theorem layer. It is also the methodological demand: if an argument can be pushed down into explicit witnesses, then it has to be pushed to bedrock; if it can still be pushed, it has to be pushed again.

Complexity implications. The classical complexity classes P and NP are defined by time cost: P is problems solvable in polynomial time; NP is problems where a solution can be *verified* in polynomial time even if finding it is hard. The Thiele Machine has a structural-cost analog: μP is the class of problems solvable with polynomial μ-cost; μNP is the class where a certified solution can be verified with polynomial μ-cost. These are formally defined in `MuComplexity.v` with basic inclusions proved — every μP problem is in μNP. Proving that $\mu\text{P} \neq \mu\text{NP}$, or finding a concrete problem in μNP that provably requires more than polynomial μ to solve, is hard research and doesn't follow from the current definitions. The factored-search witness shows a concrete structural advantage from paying $\mu = 18$, but that is a single example, not a class separation.

Shannon-style lower bound. Claude Shannon proved in 1948 that you need at least $\log_2 n$ binary questions to identify one item from a set of n items. That is the information-theoretic lower bound on search. Anything that compresses a set of n possibilities to a smaller set must gather at least that much information. The Thiele Machine analog: any feasible-set reduction (narrowing Ω to Ω') requires at least $\log_2^{\uparrow} |\Omega/|\Omega'|$ bits worth of certified structural work. `MuShannonBridge.v` proves that for any feasible-set reduction, a covering decision tree EXISTS constructively (`exists_covering_tree`). The `info_priced_reduction_no_tree_hypothesis` theorem proves the entropy bound without requiring the tree as an explicit input — the tree is chosen automatically

from the reduction sizes. The remaining open step: derive the realization condition (`decision_tree_realized_by_trace`) from arbitrary VM trace behavior without requiring the user to verify the leaf count explicitly. The naive single-trace claim is false in general and the file records that explicitly.

Quantum mechanics connection. The mathematical equivalence between Thiele honesty and zero-marginal NPA realizability is now a kernel-tier theorem (`thiele_honest_iff_quantum_realizable`, zero axioms). The zero-marginal NPA framework is the orthogonal-observables slice ($\rho_{AA} = \rho_{BB} = 0$) — a proper subset of the full quantum elliptope. Whether this restriction maps to a physically privileged class of experiments, and whether a physical Thiele Machine can produce CHSH-violating statistics by exploiting quantum entanglement in hardware, are open empirical questions outside the scope of this project.

I could pretend these are answered. I could wave at them vaguely and say "future work." I don't know. These are real open questions.

22 Conclusion

That's the whole thing.

A Turing machine is blind to global structure. It can compute anything but it has no internal account of when a computation becomes entitled to use a decomposition, invariant, morphism, or narrowed feasible set. The Thiele Machine makes that entitlement visible, first-class, and cost-bearing. No Free Insight is the first conservation law: you cannot certify stronger claims than you started with without paying in a monotone ledger μ that records every structural commitment.

The proof story is stronger than a one-line ledger rule. Classical shadows are many-to-one quotients that lose entitlement-relevant structure. Strict feasible-set narrowing can be turned into strictly stronger predicates. Certified strengthening requires structure addition. Any abstract certification system satisfying the one-step cost rule has trace-level non-free certification. And the structural-entitlement representation theorem welds the explicit witness case together: narrowing, distinguishability, certification, structure addition, and the $\Delta\mu$ lower bound. Every packaged `SoundStructuralShortcut` lands in that theorem. That is the formal spine of the project.

The machine has 46 opcodes. Most of them are ordinary compute operations that carry zero structural cost. The mandatory positive-cost certification/revelation class is LASSERT, LJOIN, EMIT, REVEAL, READ_PORT, CERTIFY, and MORPH_ASSERT. Partition and morphism-building operations can construct structure at $\delta = 0$; the paid step is the checked assertion or certification that lets the trace claim it. That is the point of No Free Insight.

The categorical morphism layer is strictly richer than any classical equivalent. Two

programs with identical registers, memory, μ, and error flag can differ in morphism structure, and no classical observer can see that difference. The Coq proof checks it.

The hardware design implements all 46 opcodes in RTL. All 46 have formal bisimulation proofs — Coq proofs that the hardware step function matches the Coq spec step-for-step (zero Admitted). 30 have fully unconditional proofs (the SupportedOpcode group). 6 more require valid instruction arguments (CALL/RET need stack-in-bounds; CHSH_TRIAL needs bit-values $\in \{0, 1\}$; TENSOR_SET/GET need valid indices; LASSERT needs declared length to match actual). 7 (the morphism group) are unconditional for any state reachable from hardware reset. 3 (PNEW, PSPLIT, PMERGE) have a semantic-level boundary where the hardware and VM represent zero-size modules differently. The gap registry is empty (`rtl_gap_count = 0`), documented in `RTLGapRegistry.v`.

The Landauer connection and the CHSH/Tsirelson result are real: one motivates the cost schedule, one is a pure algebraic theorem. Neither requires physical claims.

Thiele honesty is exactly zero-marginal NPA realizability. That biconditional is zero-axiom, zero-physics, pure polynomial arithmetic. The zero-marginal NPA framework is the orthogonal-observables slice of the quantum elliptope; the qualification is documented in `ThieleQuantumEquivalence.v` and is not a gap — it is a precise scope statement. The μ-cost hierarchy is an infinite strict separation. Every level costs what it costs, and no level can be reached for less. These are kernel-tier results that I didn't plan for and did not expect. The machine told me. Zero Inquisitor findings. Every claim has a falsification condition. If this is wrong, it will fail visibly, and that is exactly how it should be.

I built this. Read the proofs.